Friedrich-Alexander-Universität
Technische Fakultät

Hardware Acceleration of the LeNet-5 Architecture

Rechnerarchitekturen für Deep-Learning Anwendungen

	Maxim Balajan	Markus Gerber
Matrikelnummer	23228101	22972477
StudOn-Kennung	hi46pubi	ba14acym
E-Mail	maxim.balajan@fau.de	markus.gerber@fau.de

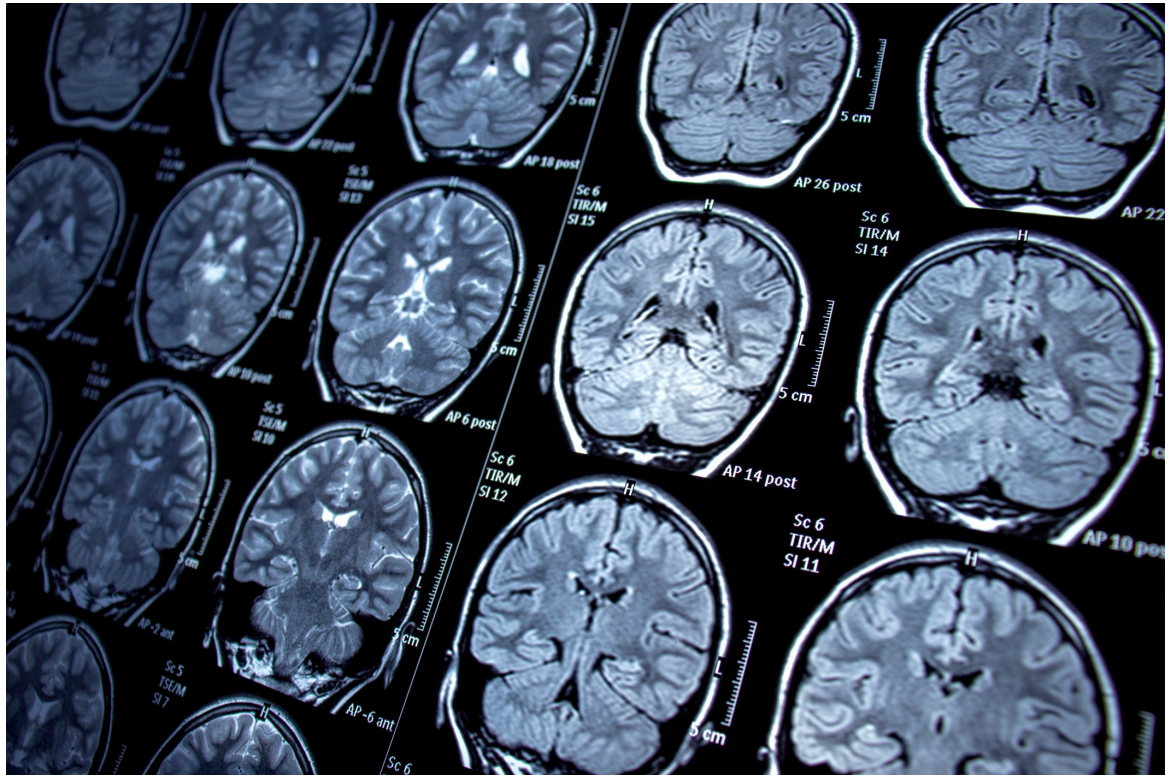
March 27, 2025



1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

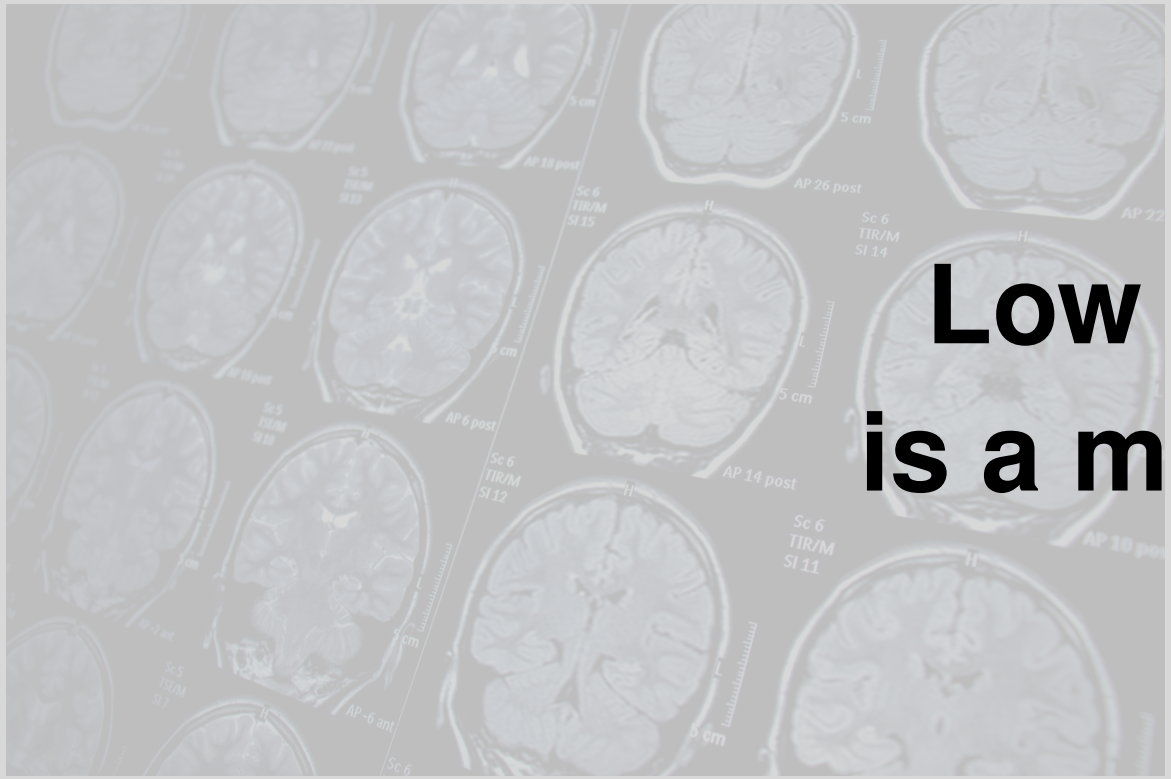
Deep-Learning and Image Recognition

Fields of application





Fields of application

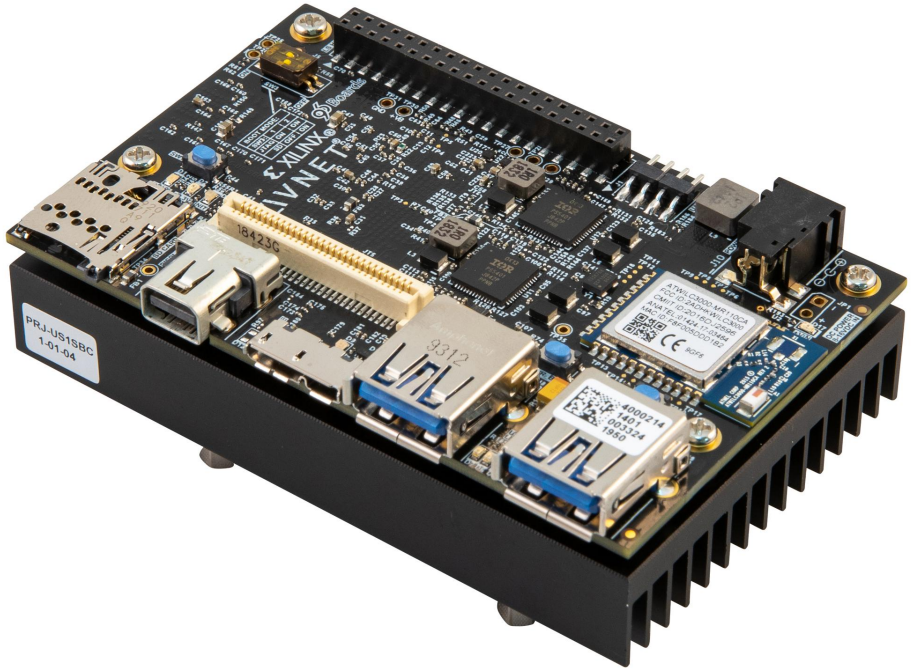




Low Latency
is a must-have!

Efficient Classification

Speed-Up via Field-Programmable-Gate-Arrays



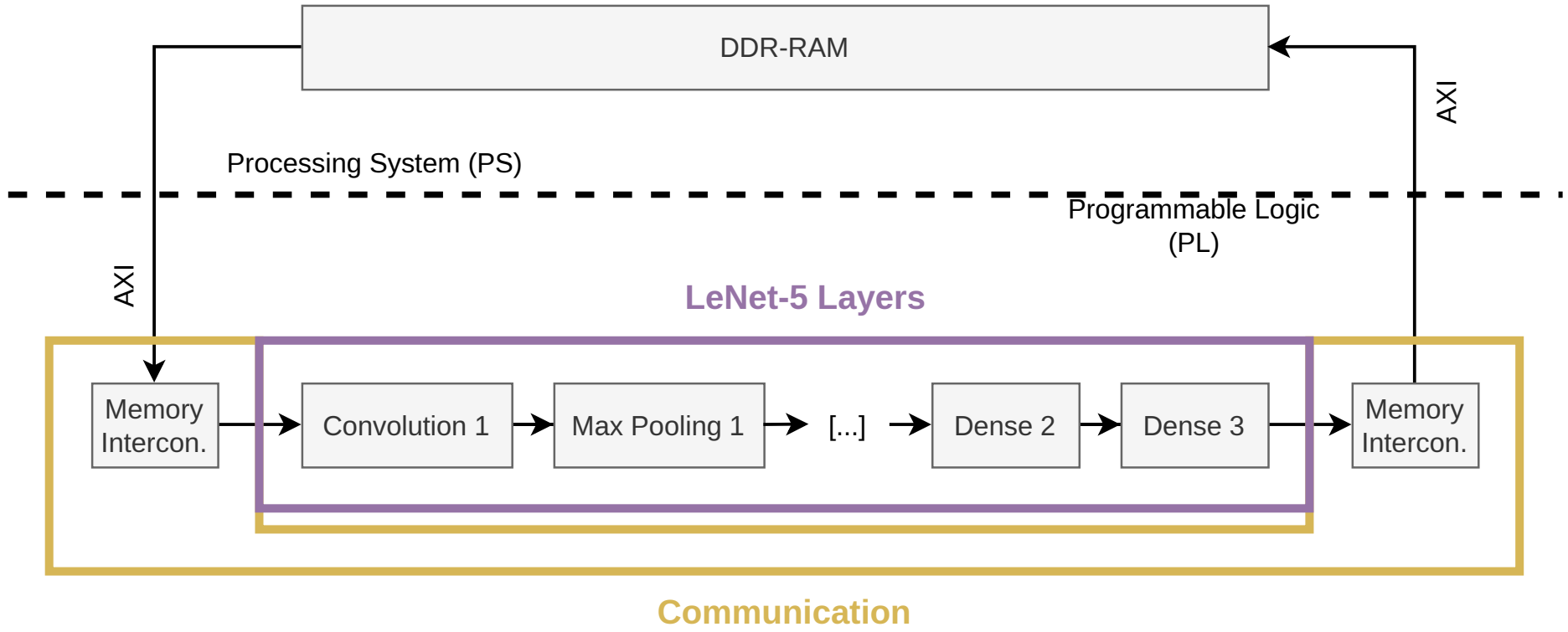
- **Board:** Ultra96-v2
- **Arm-based Zynq Ultrascale+ MPSoC:**
  - **Processing System (PS):** Quad-core ARM Cortex-A53
  - **Programmable Logic (PL):**

CLB LUTs:	70.560
CLB FFs:	141.120
Block RAM Blocks:	216

- **Application-Type:** Bare metal (no OS)

Efficient Classification

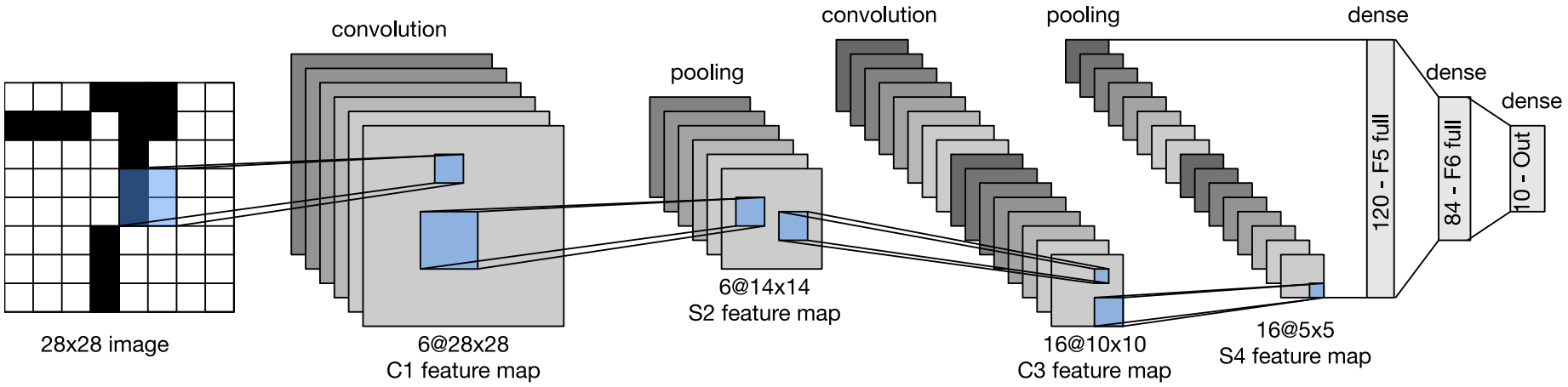
Hardware Acceleration of Convolutional-Neural-Networks



Q. H. Vo, N. Linh Le, F. Asim, L.-W. Kim, and C. S. Hong. "A Deep Learning Accelerator Based on a Streaming Architecture for Binary Neural Networks". In: *IEEE Access* 10 (2022), pp. 21141–21159

Proposed Prototype

Hardware Acceleration of the LeNet-5 Architecture



Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86 (Dec. 1998), pp. 2278–2324. DOI: 10.1109/5.726791

- Classification of handwritten digits (MNIST-Dataset) with $dim = 28 \times 28$-pixel
- *Restriction: batch_size* = 1



1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

Prerequisites

Training and Quantization of the model

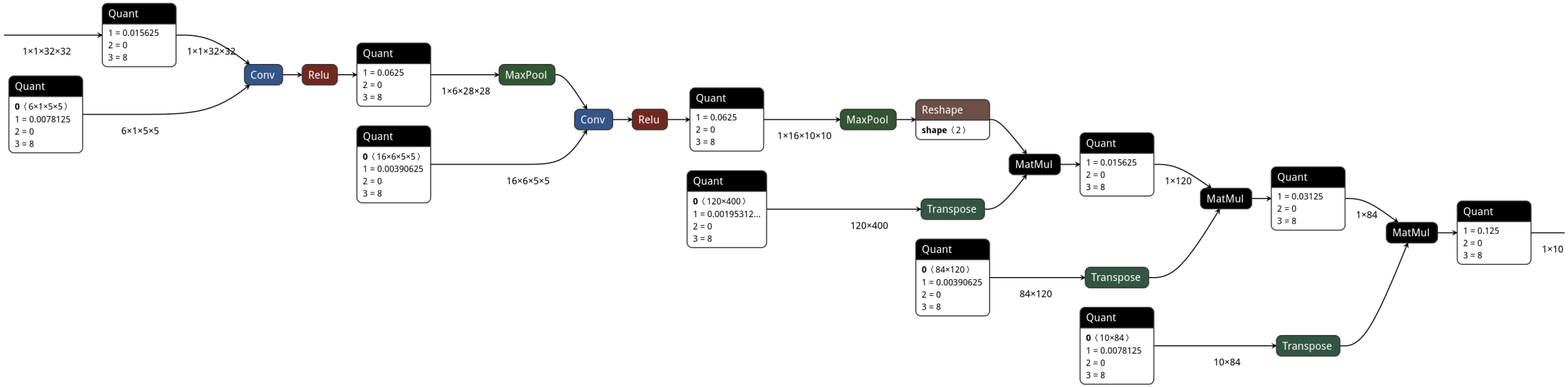


Figure: Visual representation of trained PyTorch model

- Training achieved using PyTorch and the quantization framework Brevitas
- 8-Bit Fixed-Point-Quantization used for the CNN
- Network trained over 100-Epochs with MNIST-Dataset provided by PyTorch

$\Rightarrow$ *Accuracy:* $\approx$ 98.91% (10.000 test images)

Prerequisites

Reference Implementation

Why do you need a software reference implementation for developing hardware?

1. **Optimization**: Minimize architecture & required hardware

```
// choose target types (e.g. int8_t, Int24 (custom), ClippedInt8 (custom), ...)
using MainType = ClippedInt8;
using AccumulatorType = ClippedInt24;

// parse & pre-process layers
auto layers = parse_layers<MainType, AccumulatorType>("LeNet5.onnx");

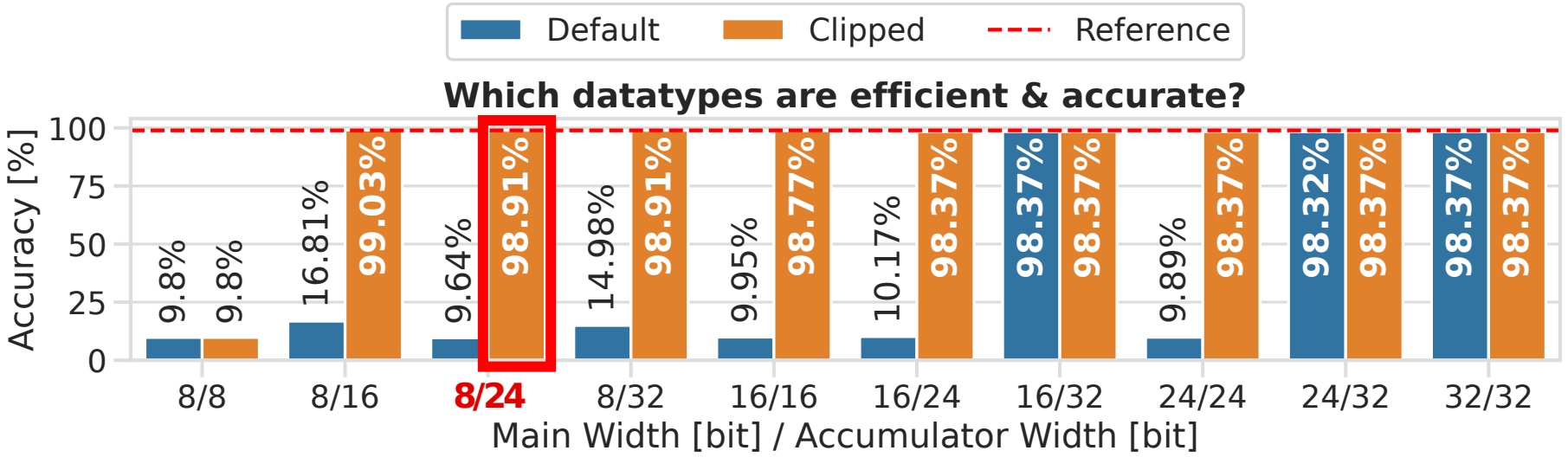
// run inference
const Matrix<MainType>* curr_input = &input;
for (auto& layer : layers) {
  curr_input = &layer.calc(*curr_input);
}
```

Prerequisites

Reference Implementation

Why do you need a software reference implementation for developing hardware?

1. **Optimization**: Minimize architecture & required hardware



⇒ Datatypes: **Clipped 8-bit** main and **24-bit** accumulator

Prerequisites

Reference Implementation

Why do you need a software reference implementation for developing hardware?

1. **Optimization**: Minimize architecture & required hardware

   ⇒ Datatypes: **Clipped 8-bit** main and **24-bit** accumulator

2. **Verification**: Verify correctness of architecture

   ⇒ The architecture works & accuracy is **98.91%** (as expected)

3. **Reference**: Quick reference for hardware development

4. **Debugging**: Exact comparison of intermediate results

5. **Benchmarking**: Reference implementation as a realistic baseline

1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

Architecture Overview

Generic Accelerator Design

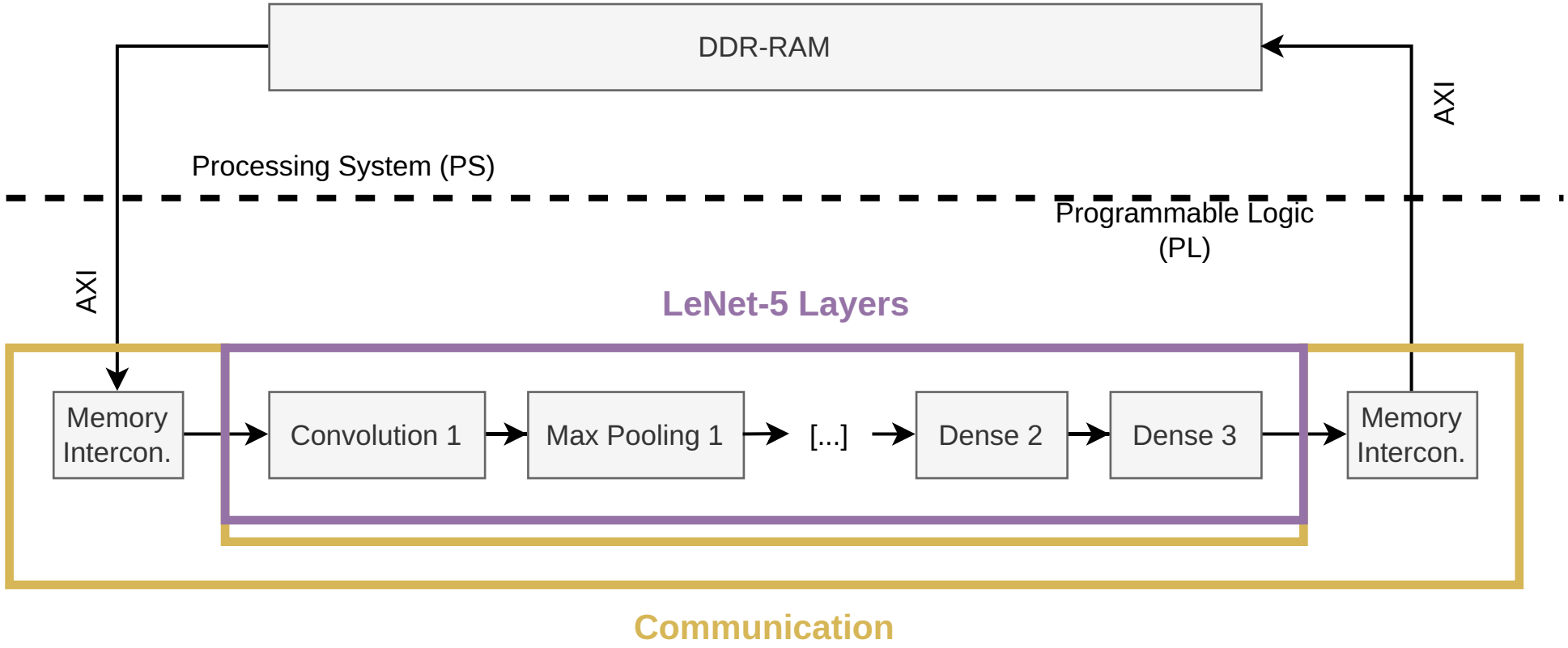
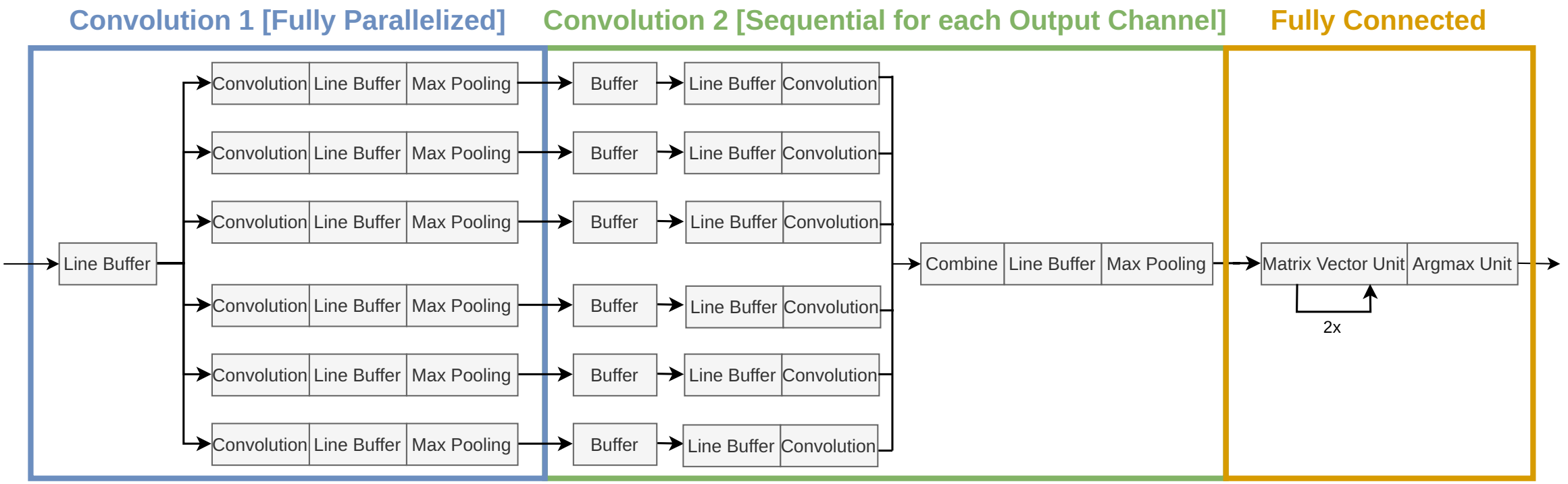


Figure: Generic accelerator structure [Vo+22]

Architecture Overview

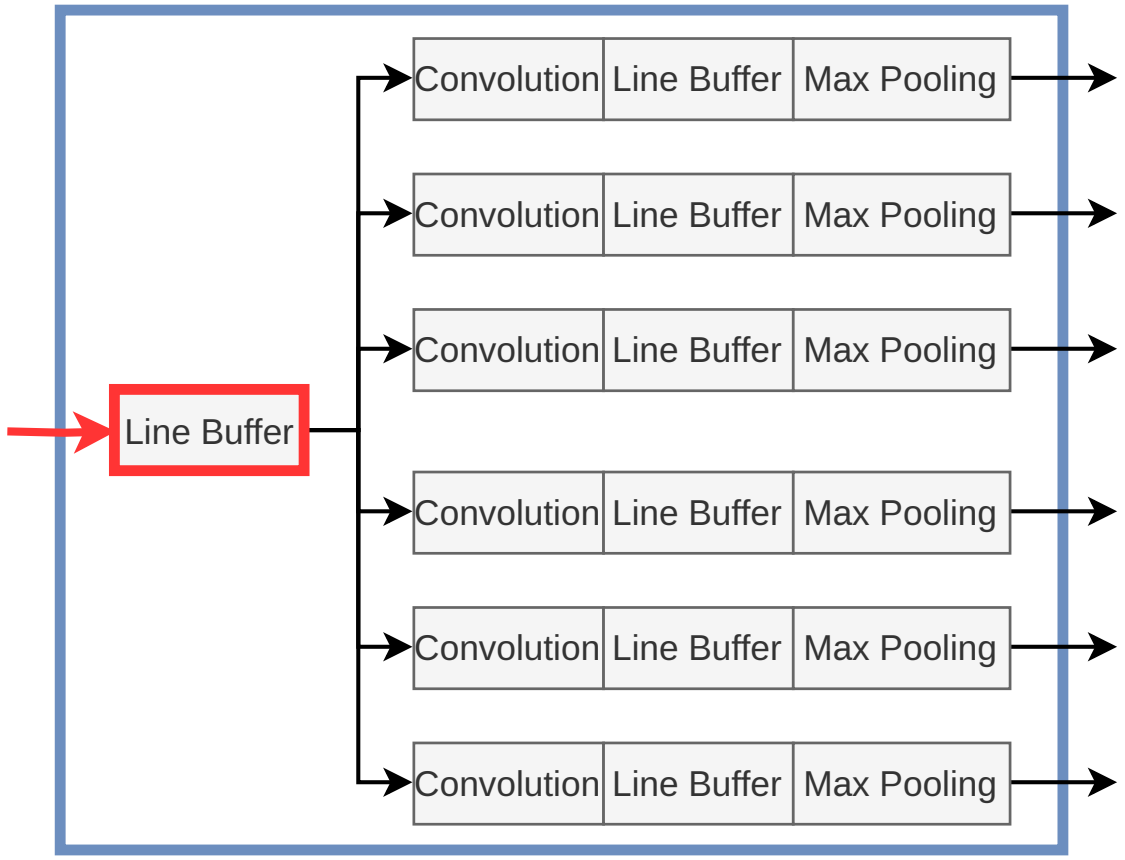
Layers of LeNet-5 Architecture



Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]

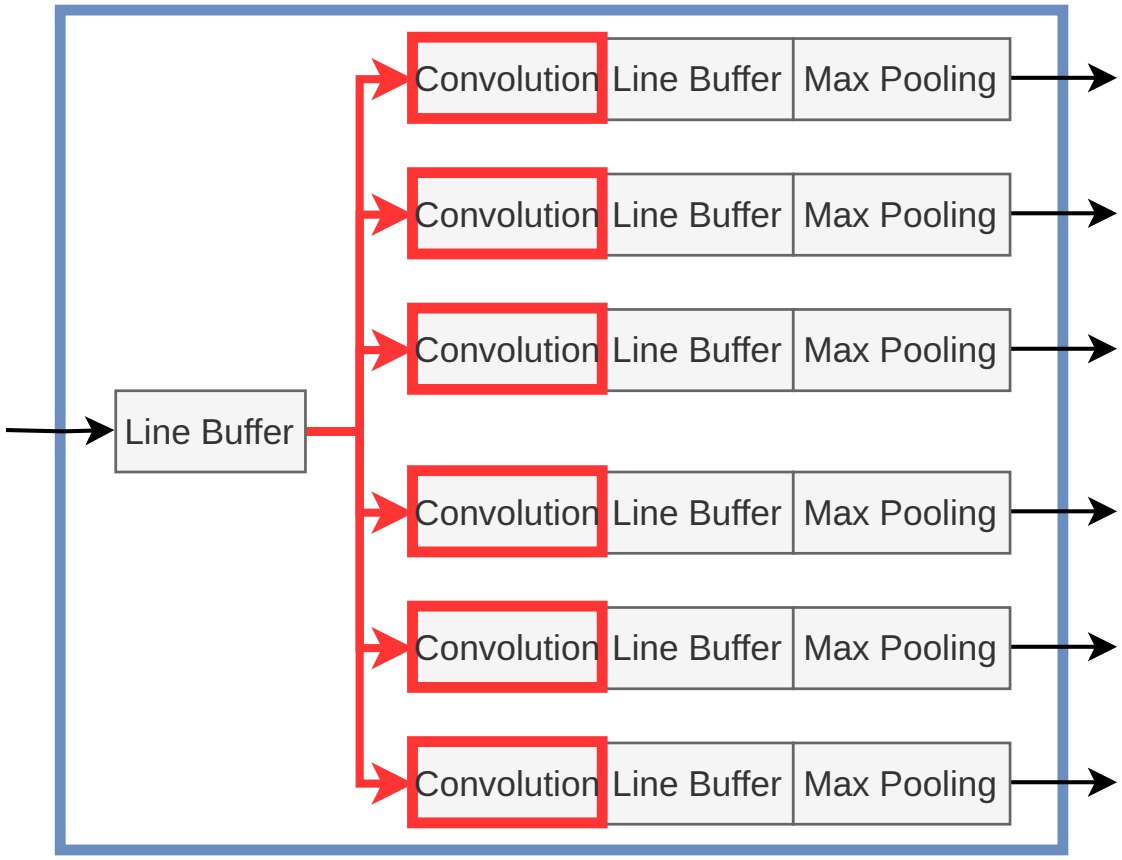


- Pixel data streamed from PS into first Linebuffer

Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]



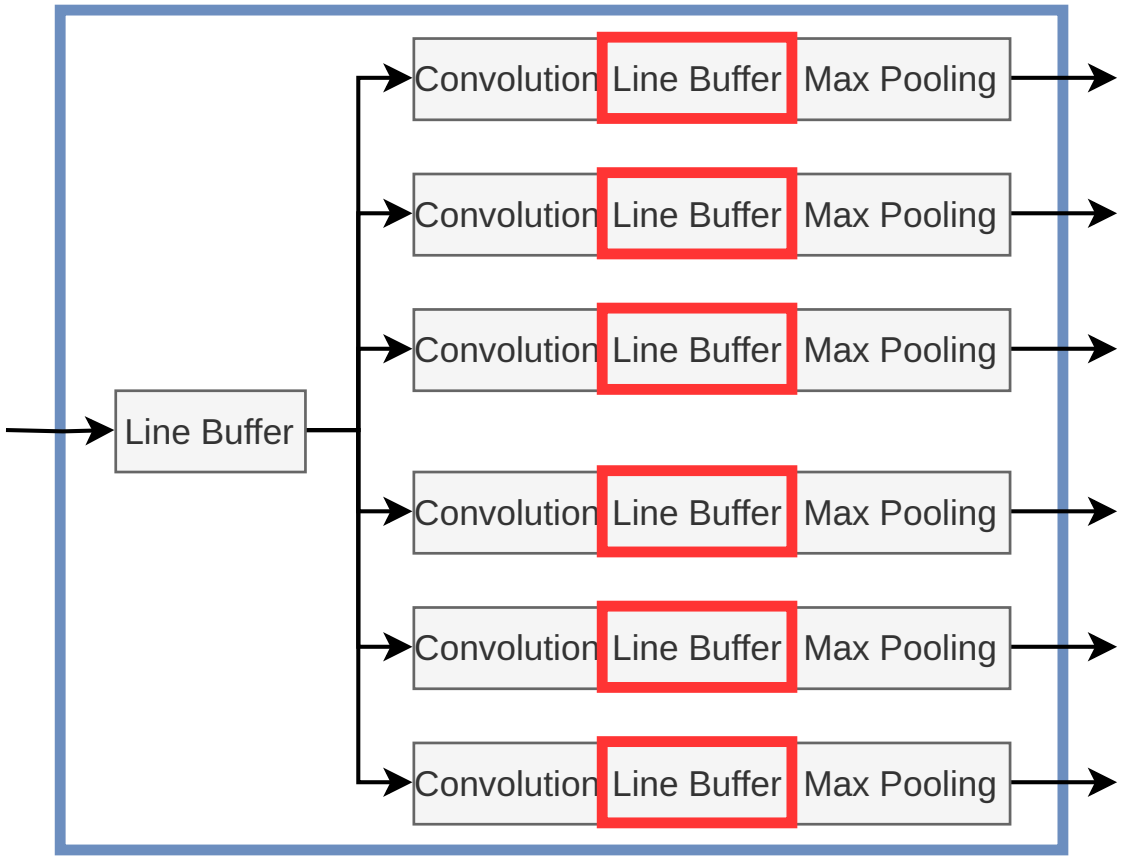
- Pixel data streamed from PS into first Linebuffer

$\Rightarrow$ Convolution of 5×5-windows with weight data

Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]



- Pixel data streamed from PS into first Linebuffer

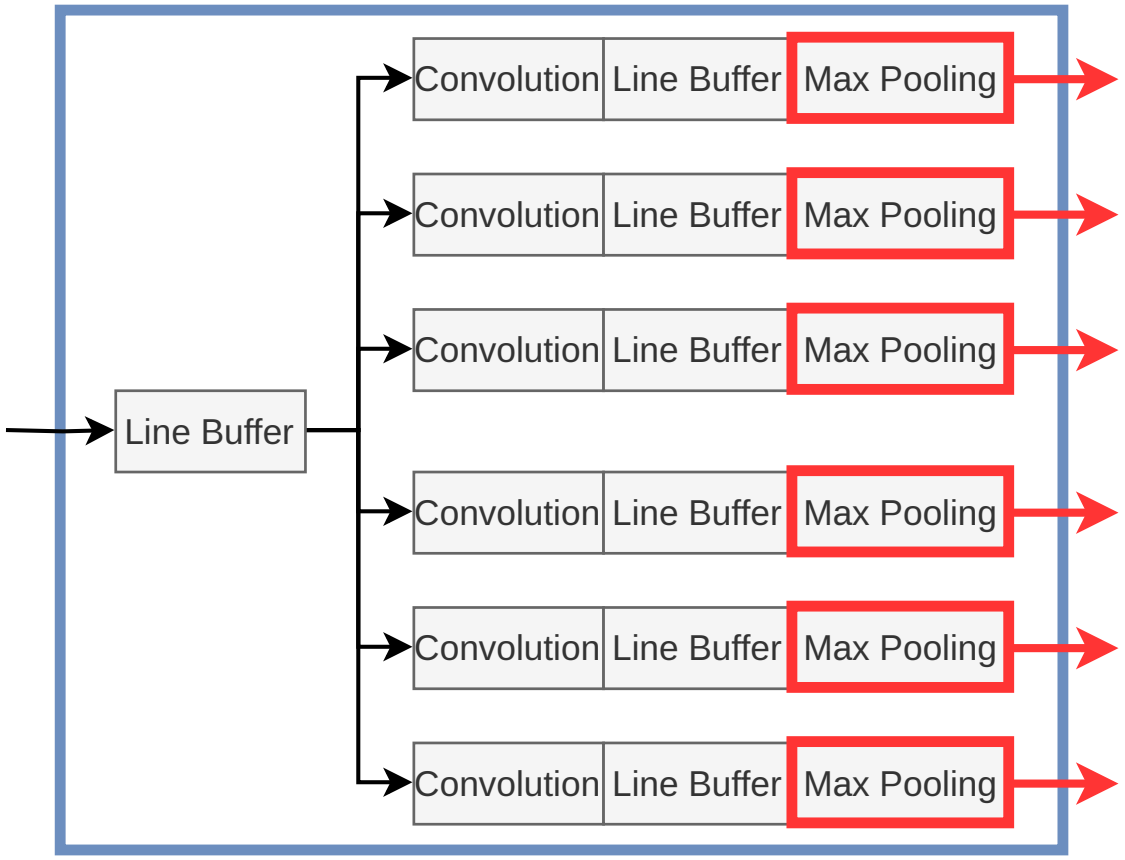
$\Rightarrow$ Convolution of 5×5-windows with weight data

- New pixel data streamed into second Linebuffer

Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]



- Pixel data streamed from PS into first Linebuffer

$\Rightarrow$ Convolution of 5×5-windows with weight data

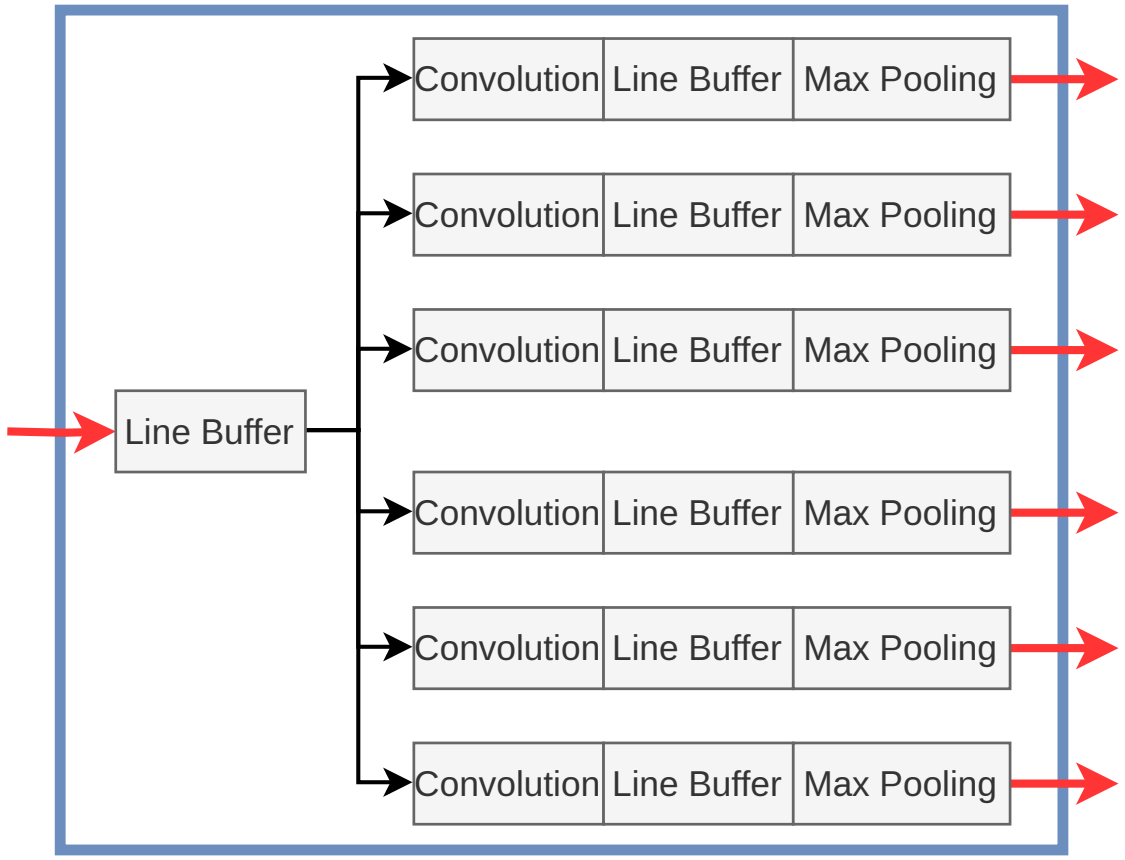
- New pixel data streamed into second Linebuffer

$\Rightarrow$ Pooling of 2×2-windows

Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]



- Pixel data streamed from PS into first Linebuffer

$\Rightarrow$ Convolution of 5×5-windows with weight data

- New pixel data streamed into second Linebuffer

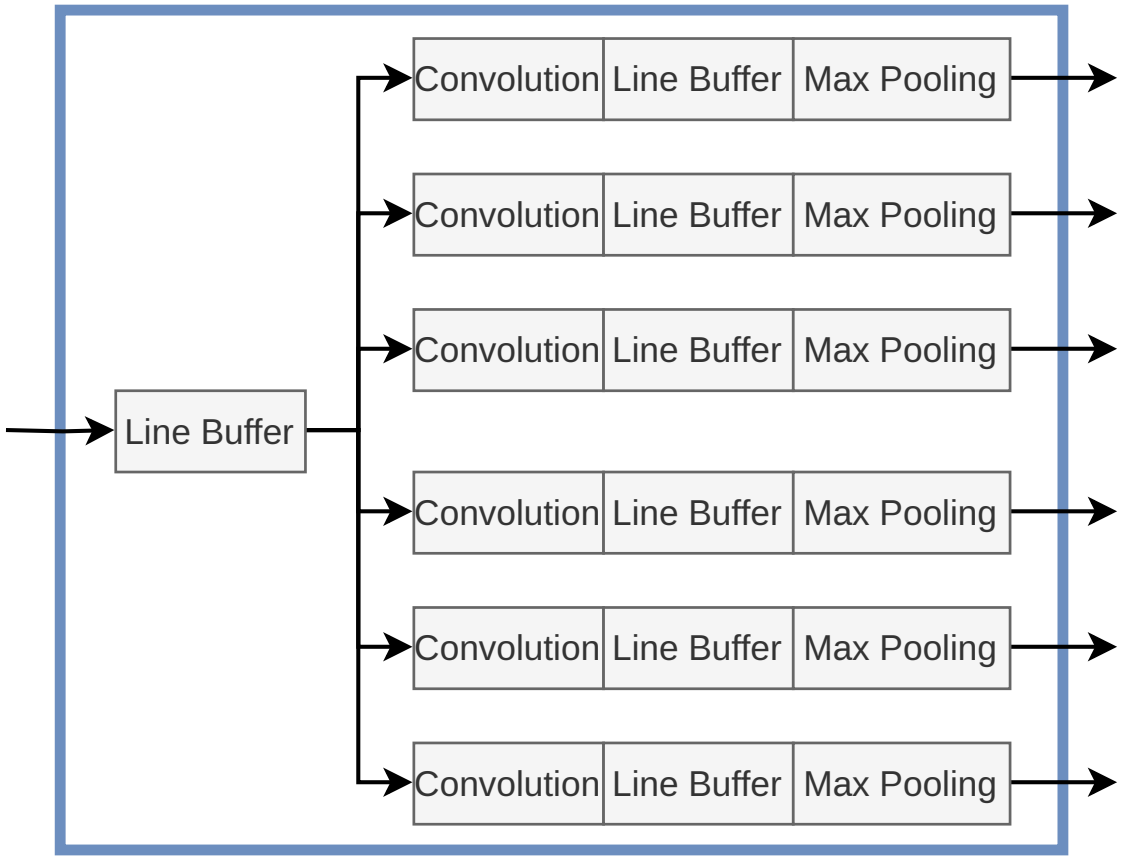
$\Rightarrow$ Pooling of 2×2-windows

- **One** input and **six** output channels

Architecture Overview

First Convolution-Phase

Convolution 1 [Fully Parallelized]



- Pixel data streamed from PS into first Linebuffer

⇒ Convolution of 5×5-windows with weight data

- New pixel data streamed into second Linebuffer

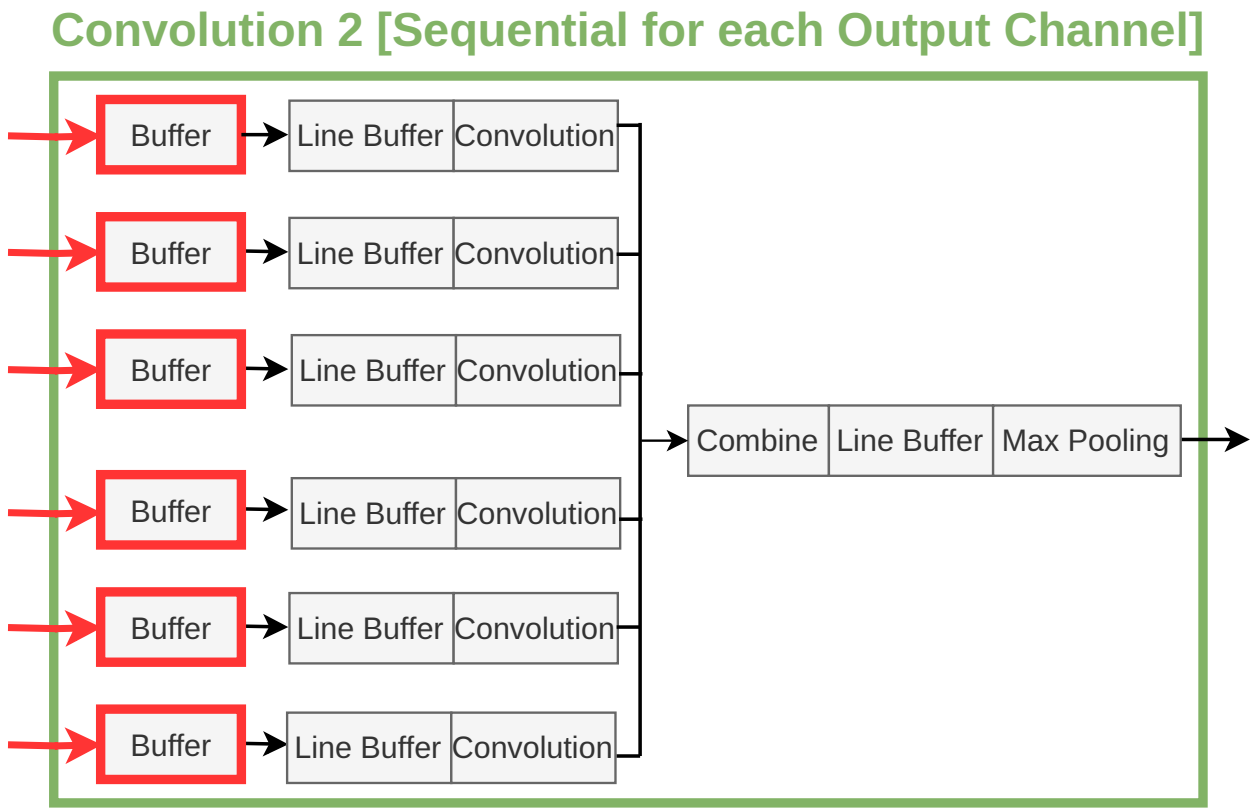
⇒ Pooling of 2×2-windows

- **One** input and **six** output channels

⇒ Six convolutions in total easily fully parallelizable

Architecture Overview

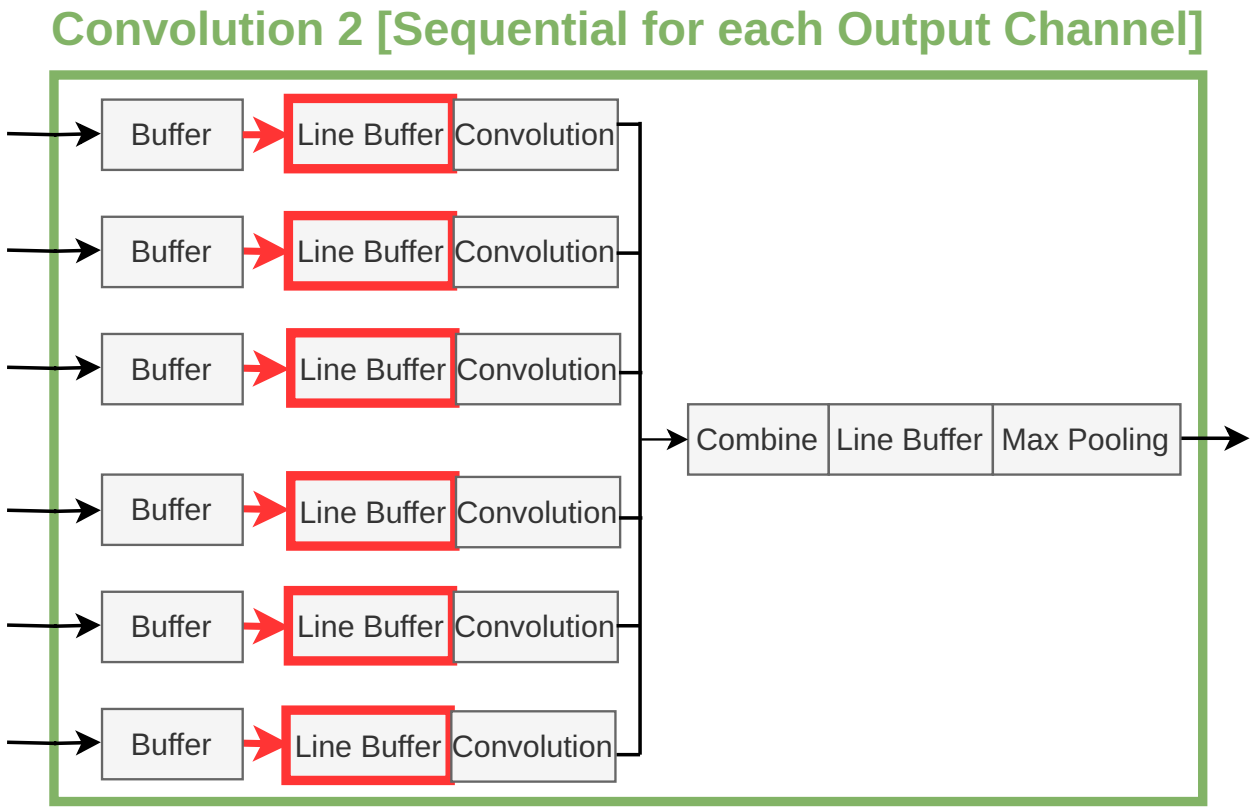
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer

Architecture Overview

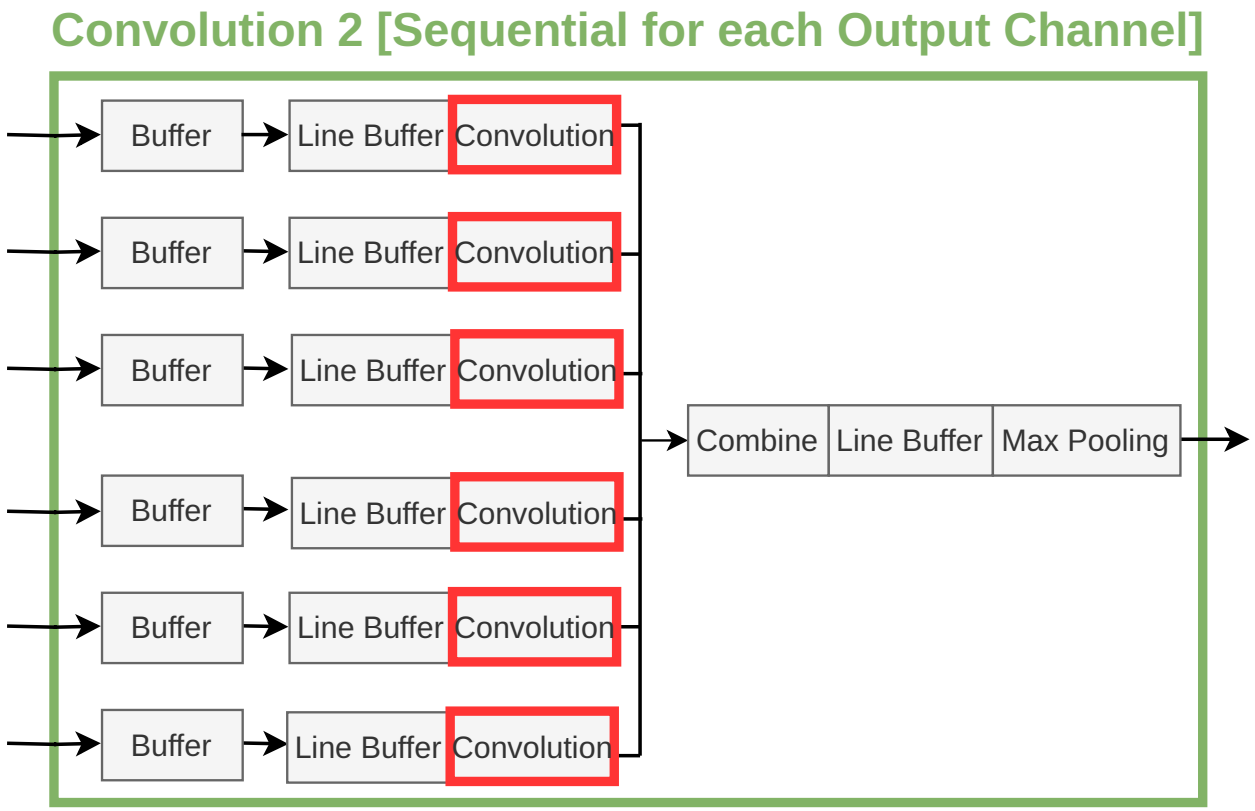
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution

Architecture Overview

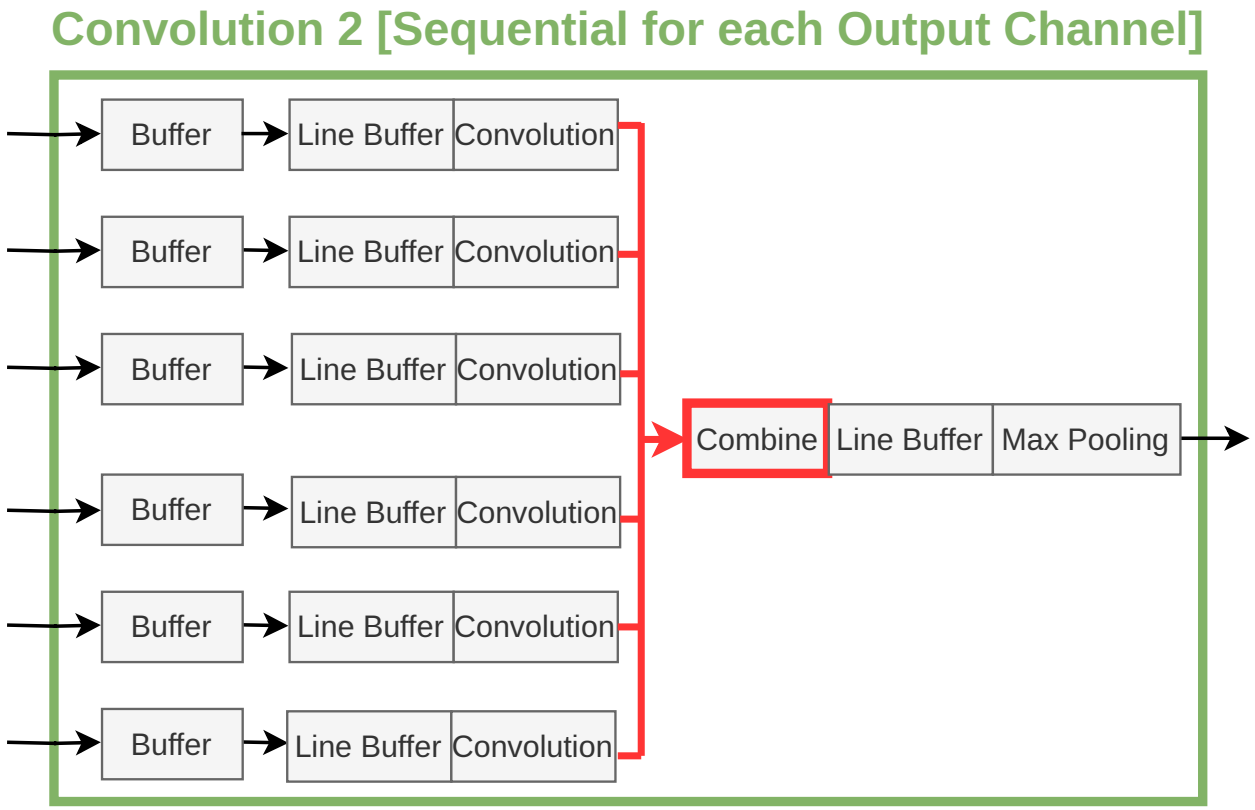
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel

Architecture Overview

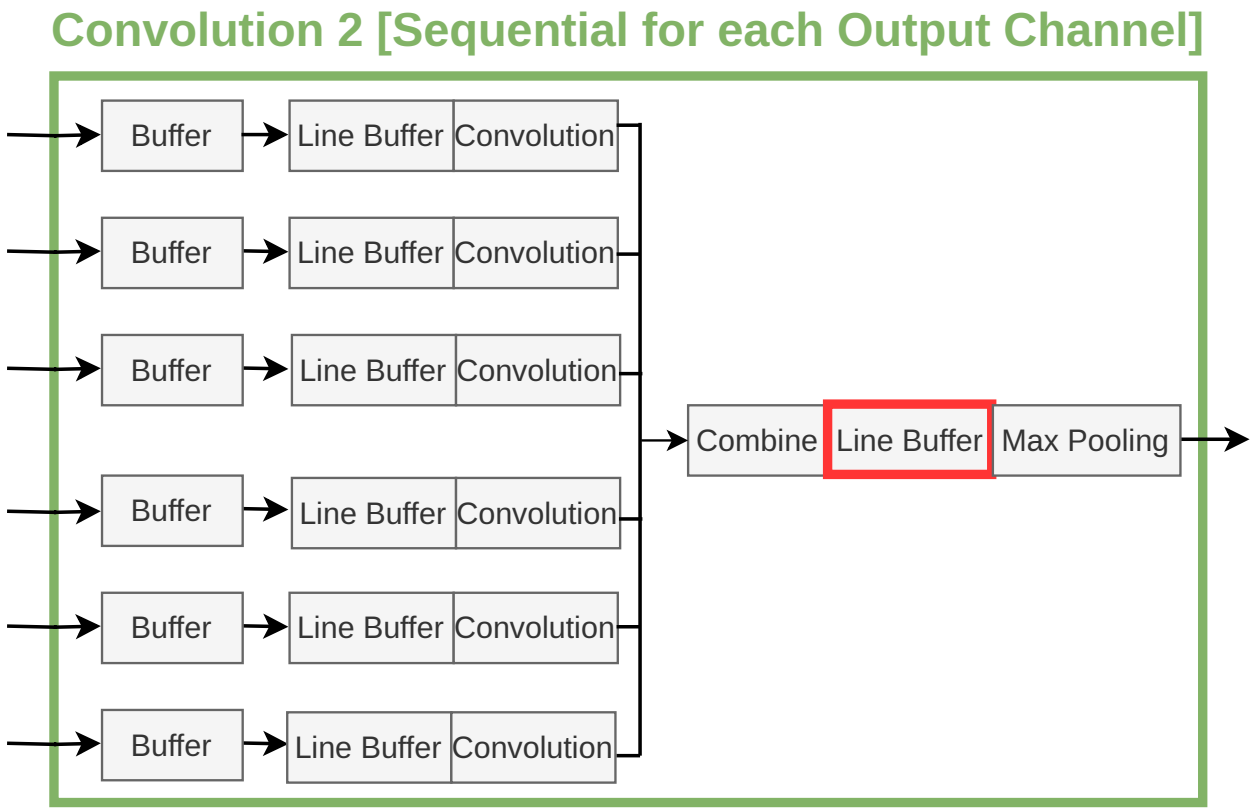
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel

Architecture Overview

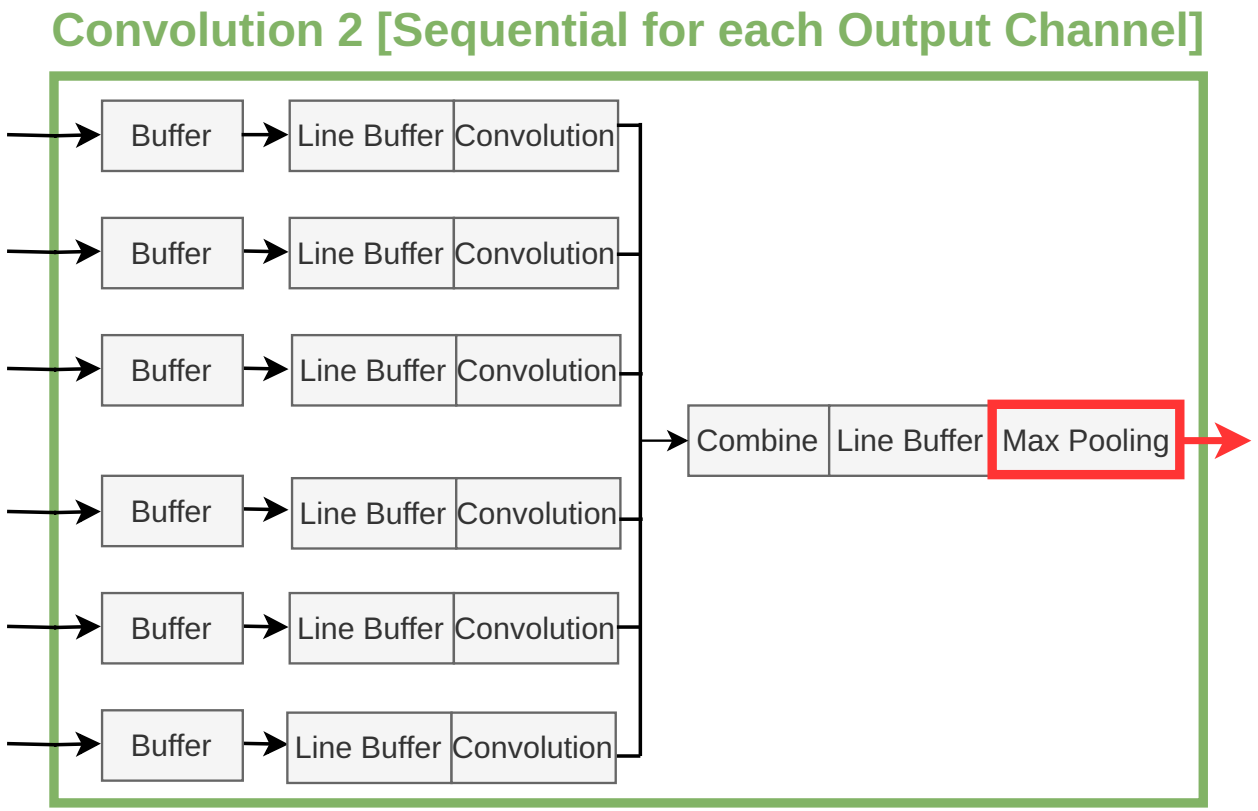
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel
- Passed to Linebuffer and Max Pooling

Architecture Overview

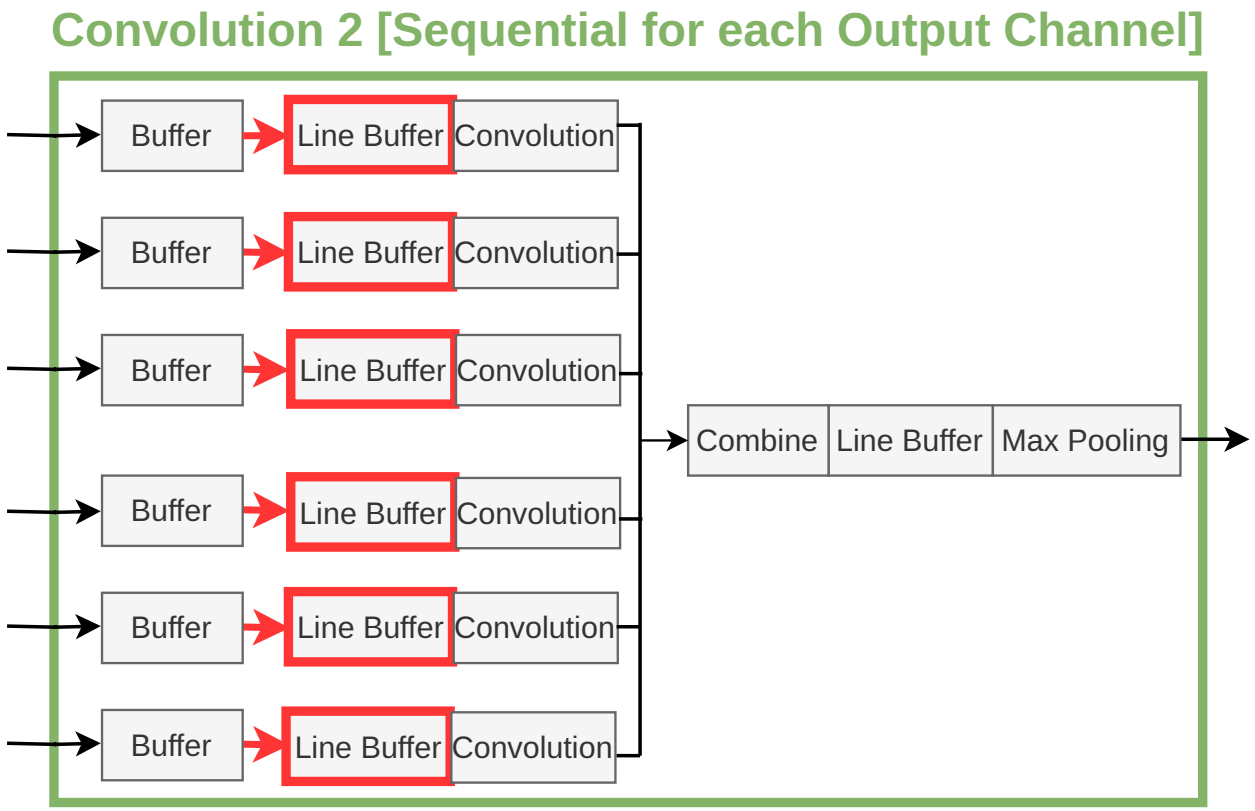
Second Convolution-Phase



- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel
- Passed to Linebuffer and Max Pooling
- First output channel finished

Architecture Overview

Second Convolution-Phase

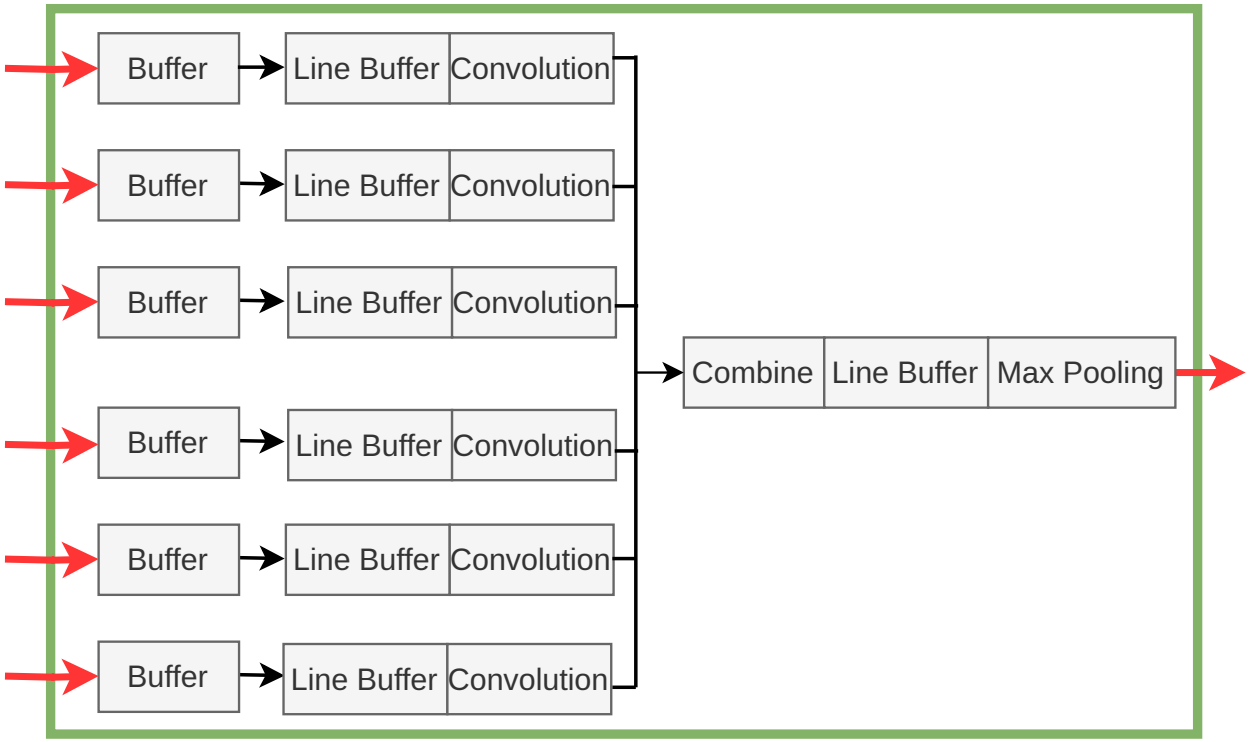


- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel
- Passed to Linebuffer and Max Pooling
- First output channel finished
- Input channels repeated with changed weights
- (...)

Architecture Overview

Second Convolution-Phase

Convolution 2 [Sequential for each Output Channel]

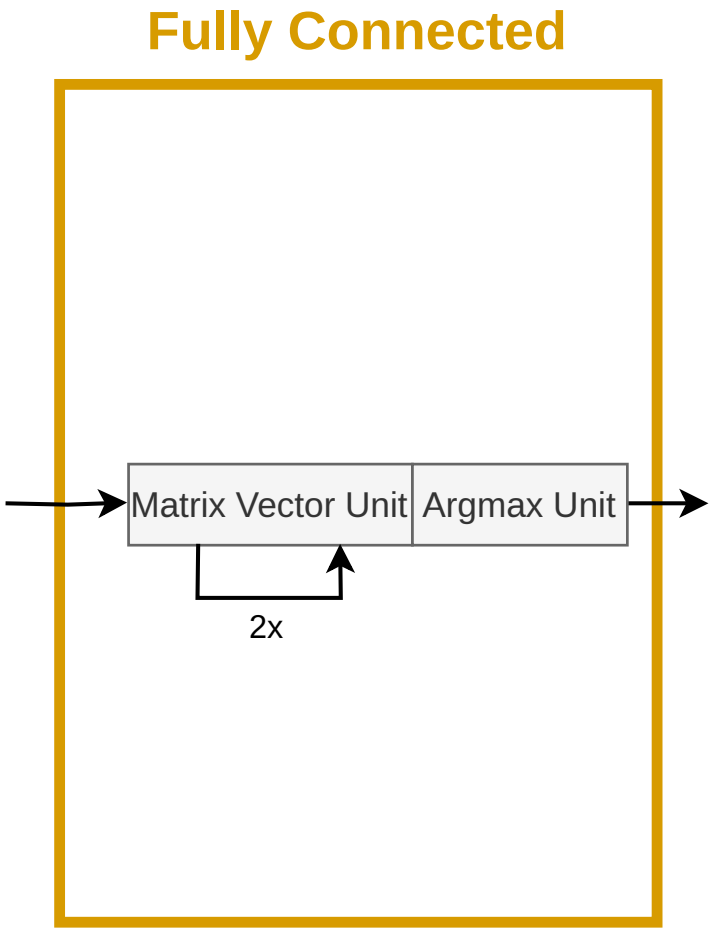


- Input channel streamed from Conv 1 into Buffer
- Passed to Linebuffer and Convolution
- Combined into one output channel
- Passed to Linebuffer and Max Pooling
- First output channel finished
- Input channels repeated with changed weights
- (...)

⇒ Compute each output channel sequentially

Architecture Overview

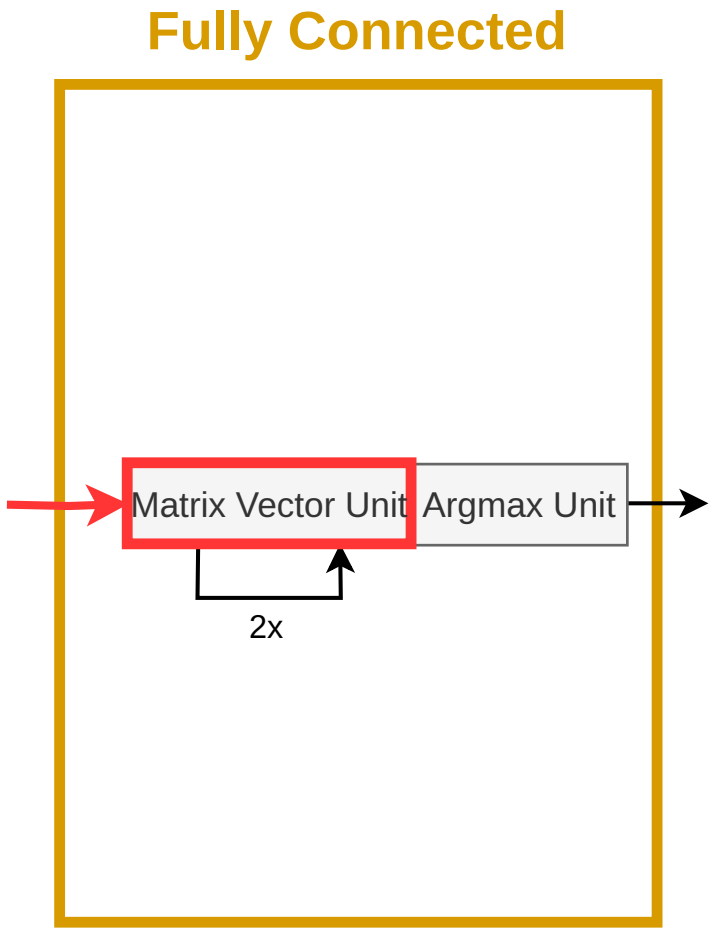
Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:

Architecture Overview

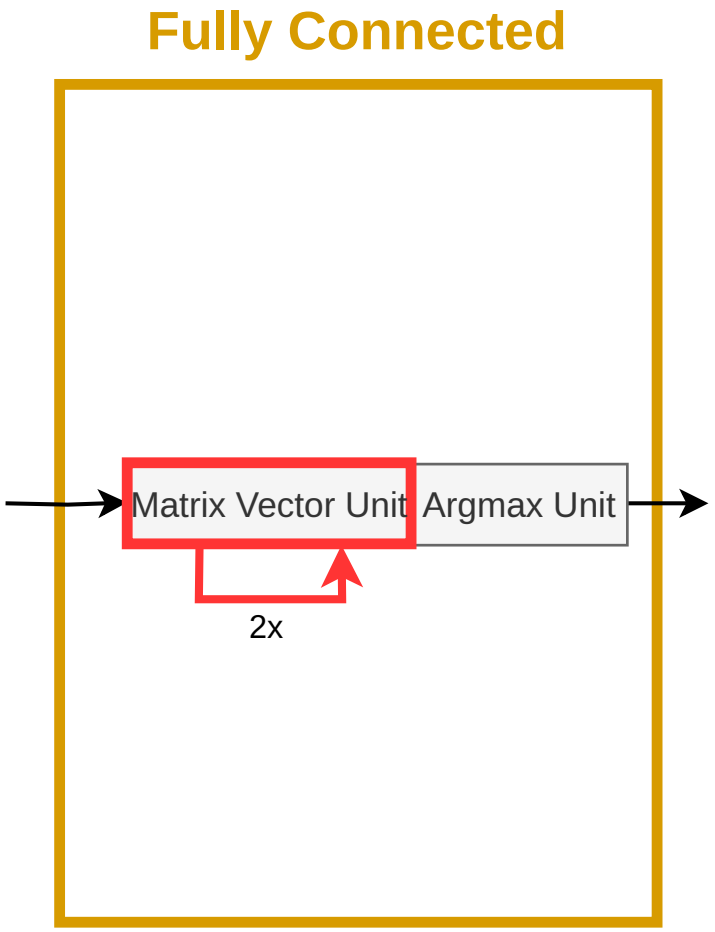
Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:
  1. Data streamed into Matrix-Vector-Unit

Architecture Overview

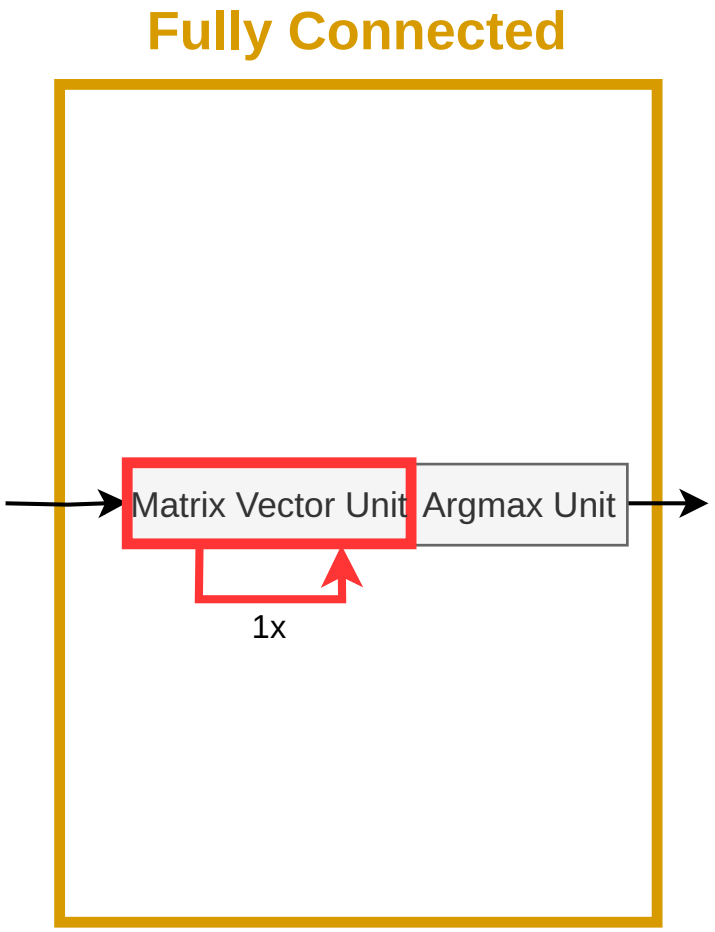
Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:
  1. Data streamed into Matrix-Vector-Unit
  2. Intermediate Results streamed back

Architecture Overview

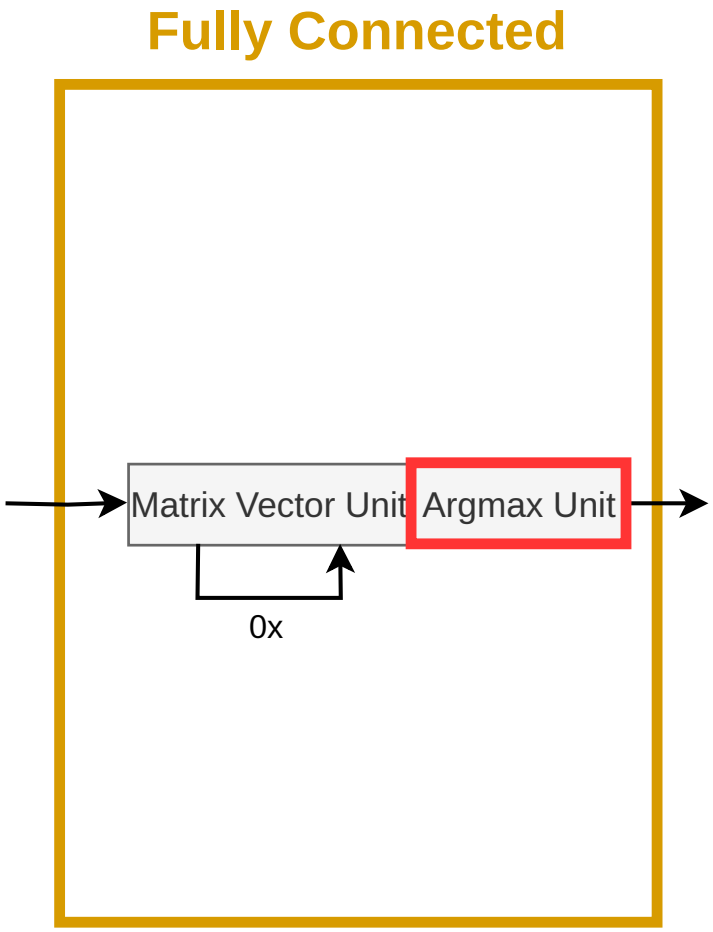
Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:
  1. Data streamed into Matrix-Vector-Unit
  2. Intermediate Results streamed back
  3. Repetition of second iteration

Architecture Overview

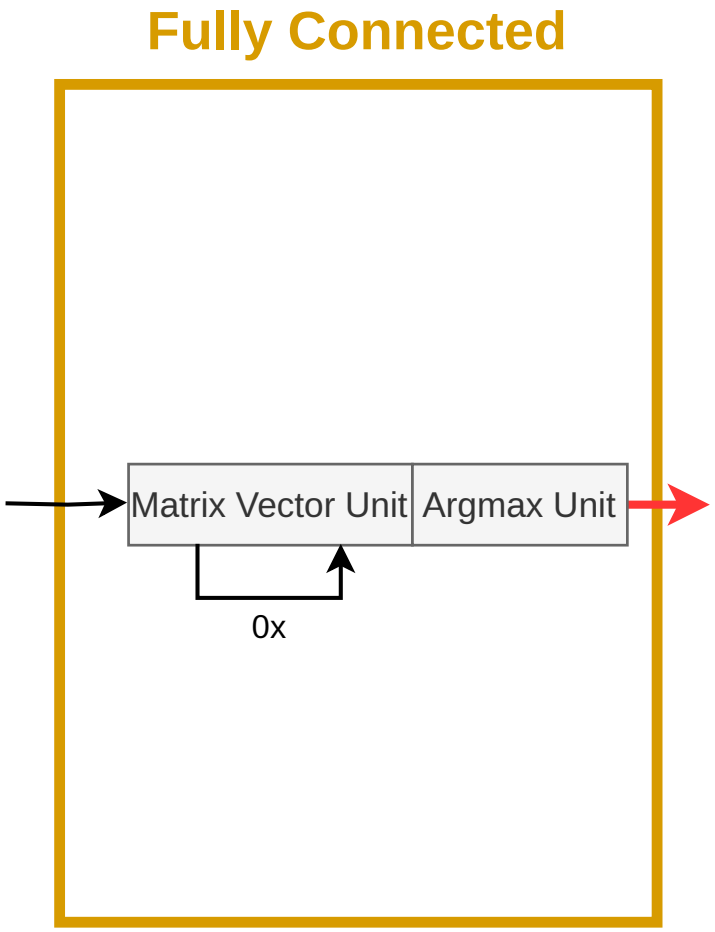
Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:
  1. Data streamed into Matrix-Vector-Unit
  2. Intermediate Results streamed back
  3. Repetition of second iteration
- 10 final label % streamed through Argmax-Unit

Architecture Overview

Fully-Connected-Phases and Argmaxing



- Data processed over **3** iterations:
  1. Data streamed into Matrix-Vector-Unit
  2. Intermediate Results streamed back
  3. Repetition of second iteration
- 10 final label % streamed through Argmax-Unit

⇒ Actual label streamed back to PS

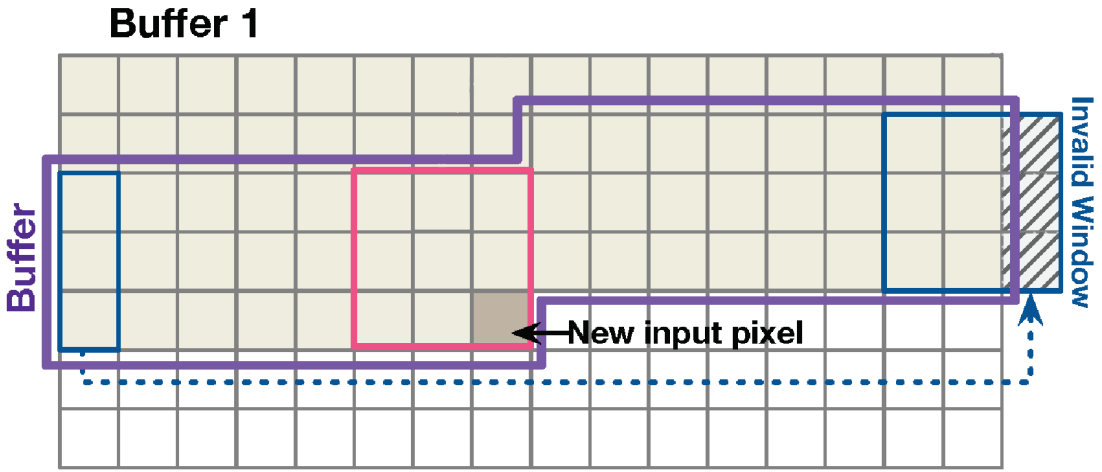
1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

Implementation

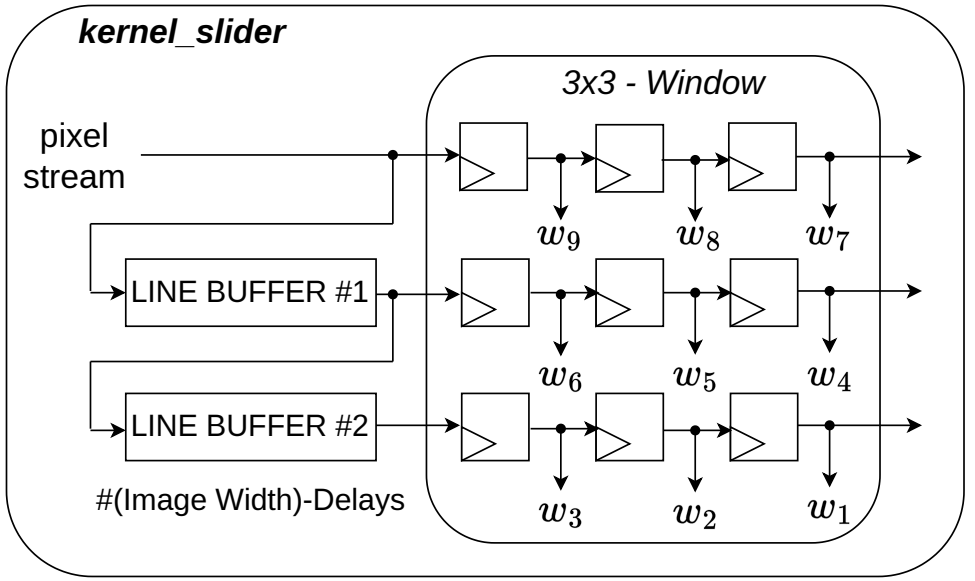
Convolutional-Layers

Kernel-Slider (Linebuffer)

Retrieving valid Kernel-Windows



R. Shi, J. S. Wong, and H. K.-H. So. "High-Throughput Line Buffer Microarchitecture for Arbitrary Sized Streaming Image Processing". In: *Journal of Imaging* 5.3 (2019). DOI: 10.3390/jimaging5030034



L. Aranda, P. Reviriego, and J. A. Maestro. "Protecting Image Processing Pipelines against Configuration Memory Errors in SRAM-Based FPGAs". In: *Electronics* 7 (Nov. 2018), p. 322

Goal:

- Stream pixels one at a time into unit

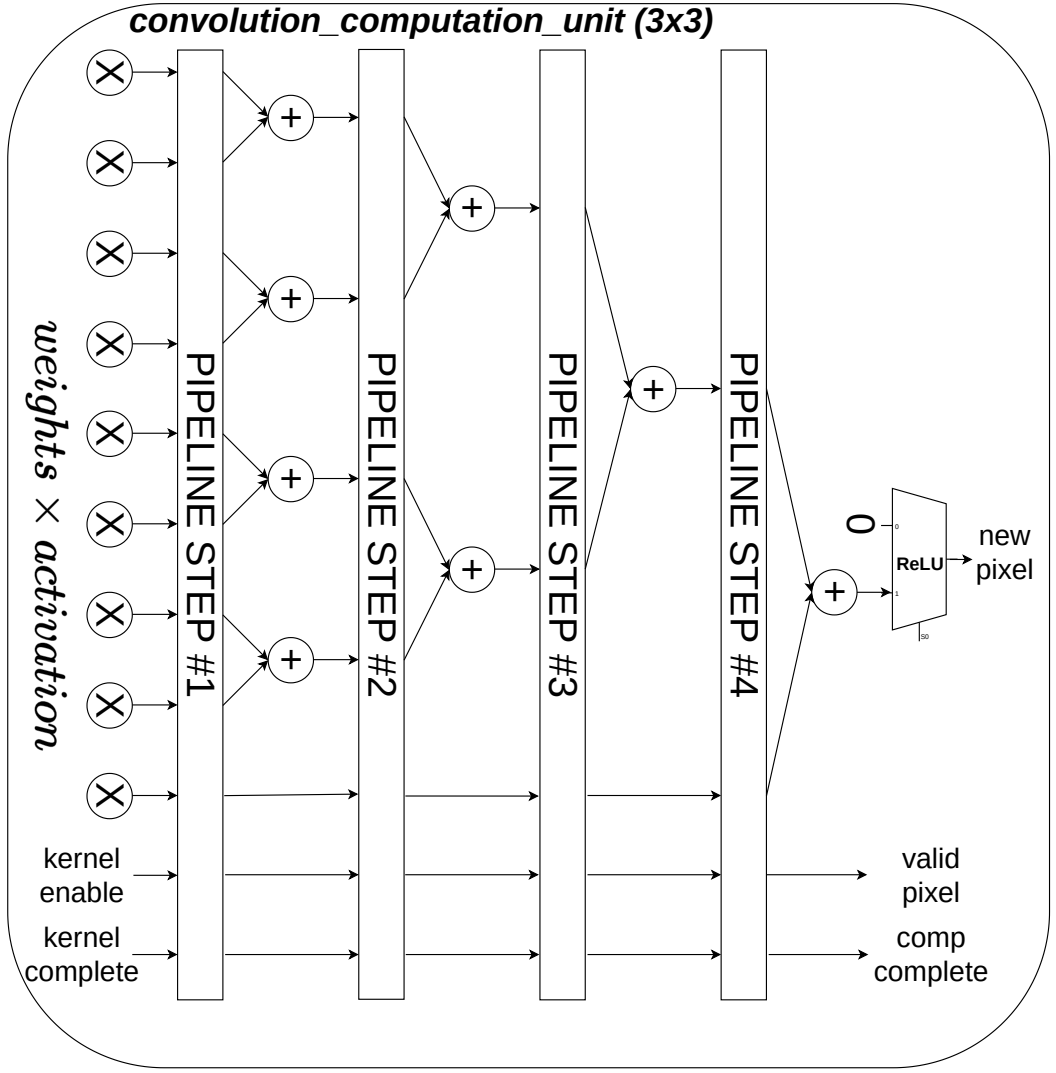
⇒ Get all valid Kernel-Windows every other cycle

- Control logic completely neglected in schematic

⇒ Separate Logic for Convolution and Pooling necessary!

Convolution-Computation-Unit

Convolution with customizability



- **Purpose:** Convolution of weights and activations
- Fully pipelined to keep Critical-Path to a minimum
- Certain pipeline steps fully customizable via generics

⇒ Control signals also pipelined for later stages

- Structure dynamically created before instantiation

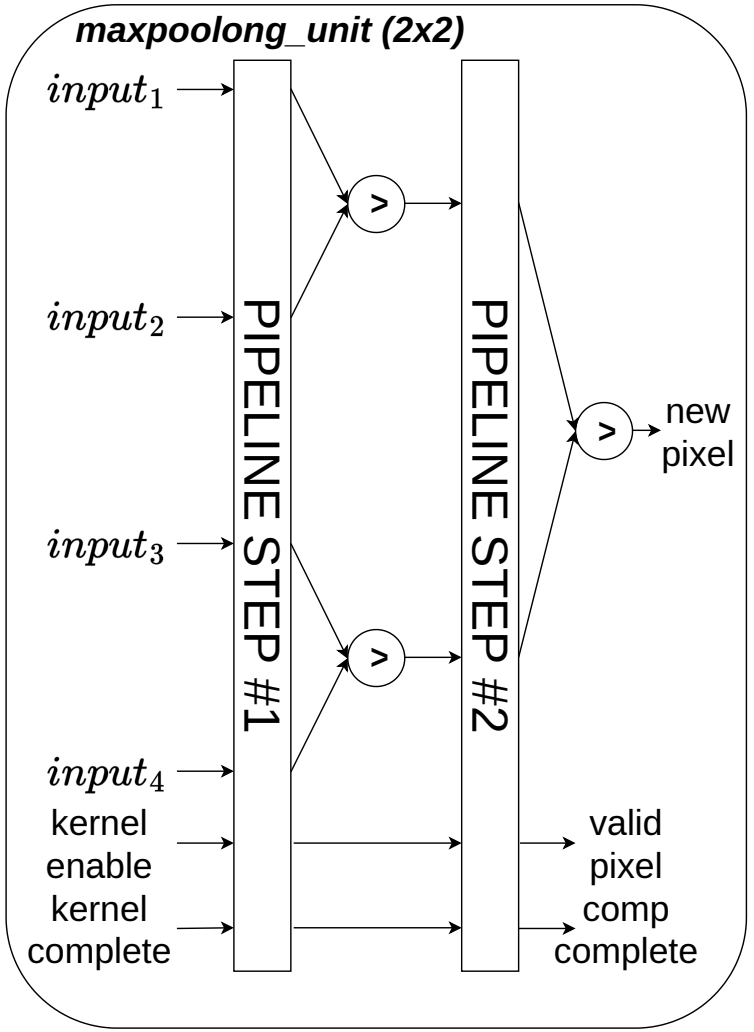
Pooling-Computation-Unit

Pooling with customizability

- **Purpose:** MaxPooling of all input values
- Fully pipelined to keep Critical-Path to a minimum

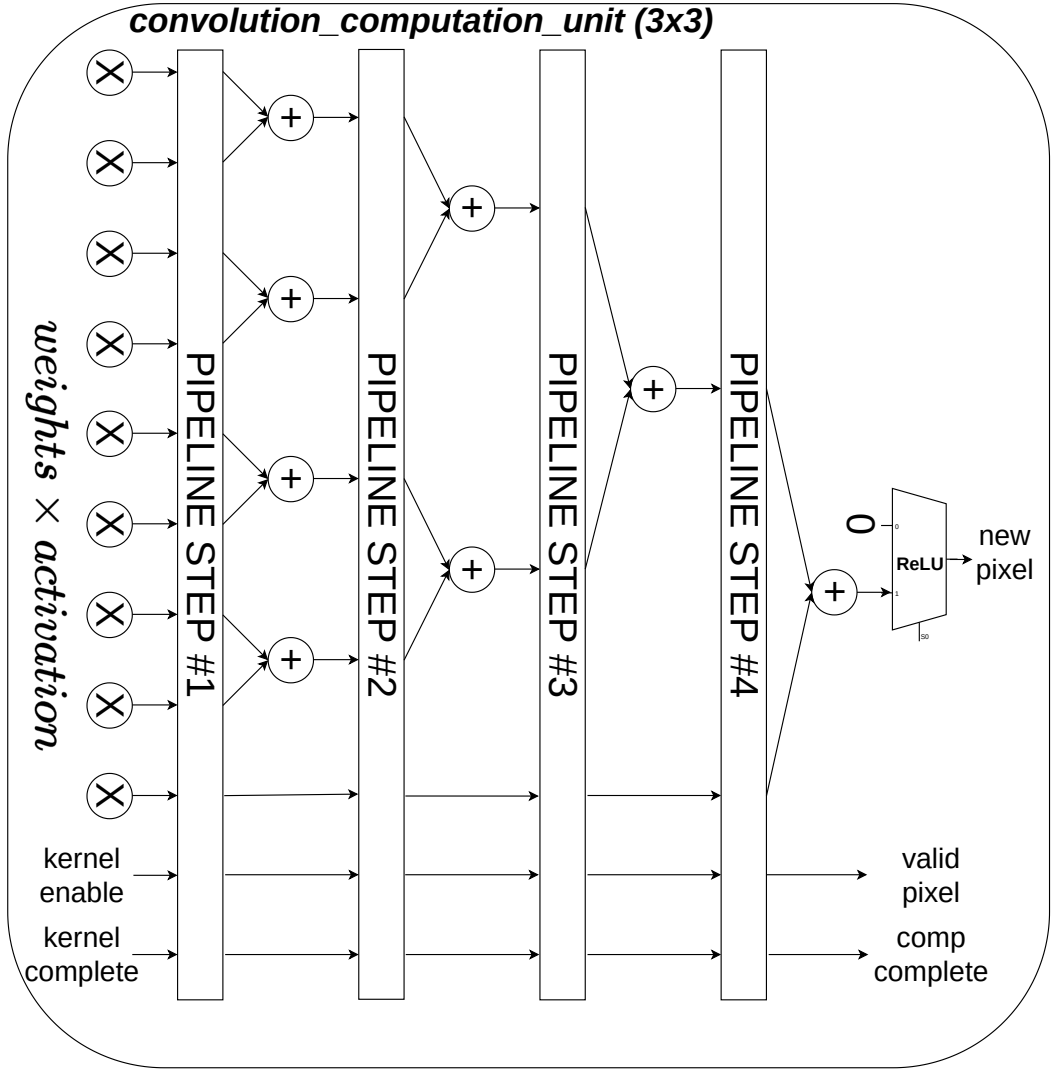
⇒ Control signals also needed for later stages

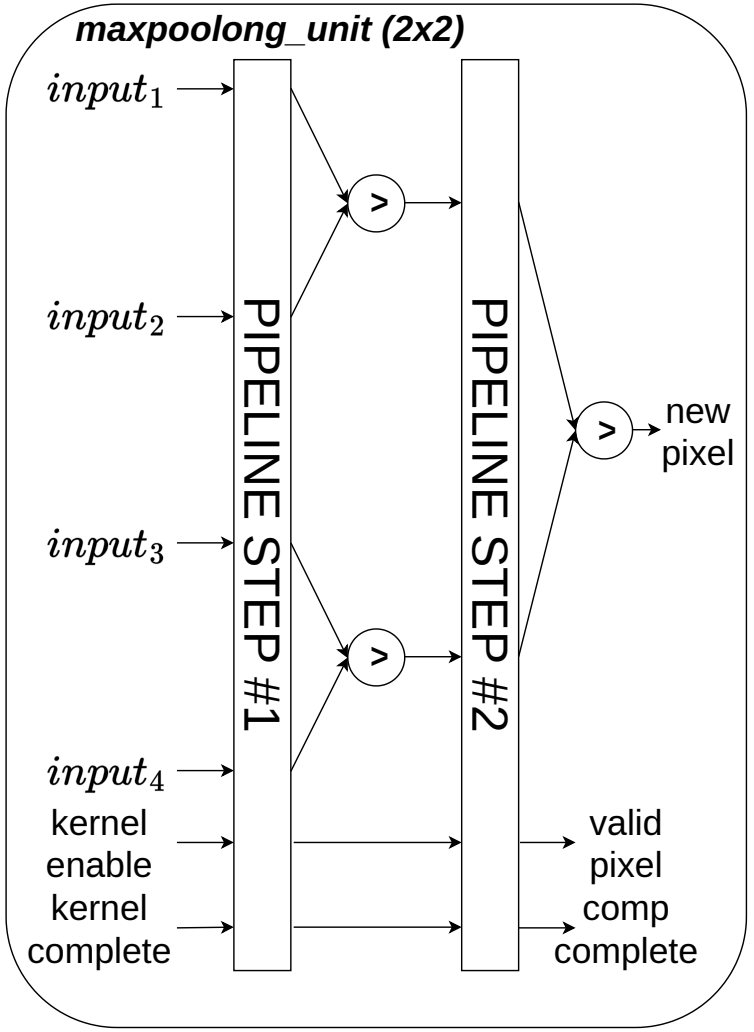
- Structure dynamically created before instantiation



Convolution-Pooling-Pipeline

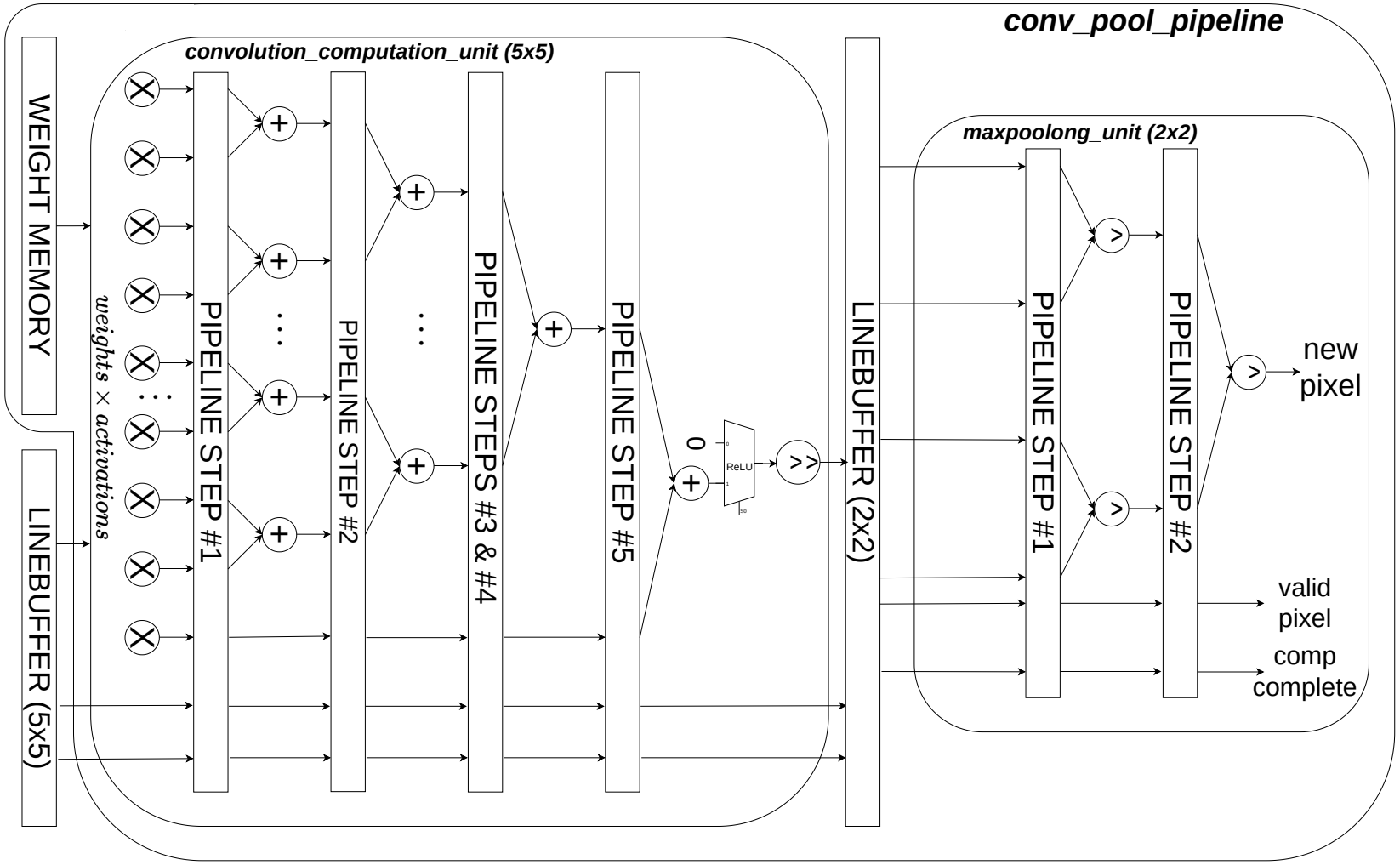
Main pipeline for realizing both Convolution-Phases





Convolution-Pooling-Pipeline

Main pipeline for realizing both Convolution-Phases

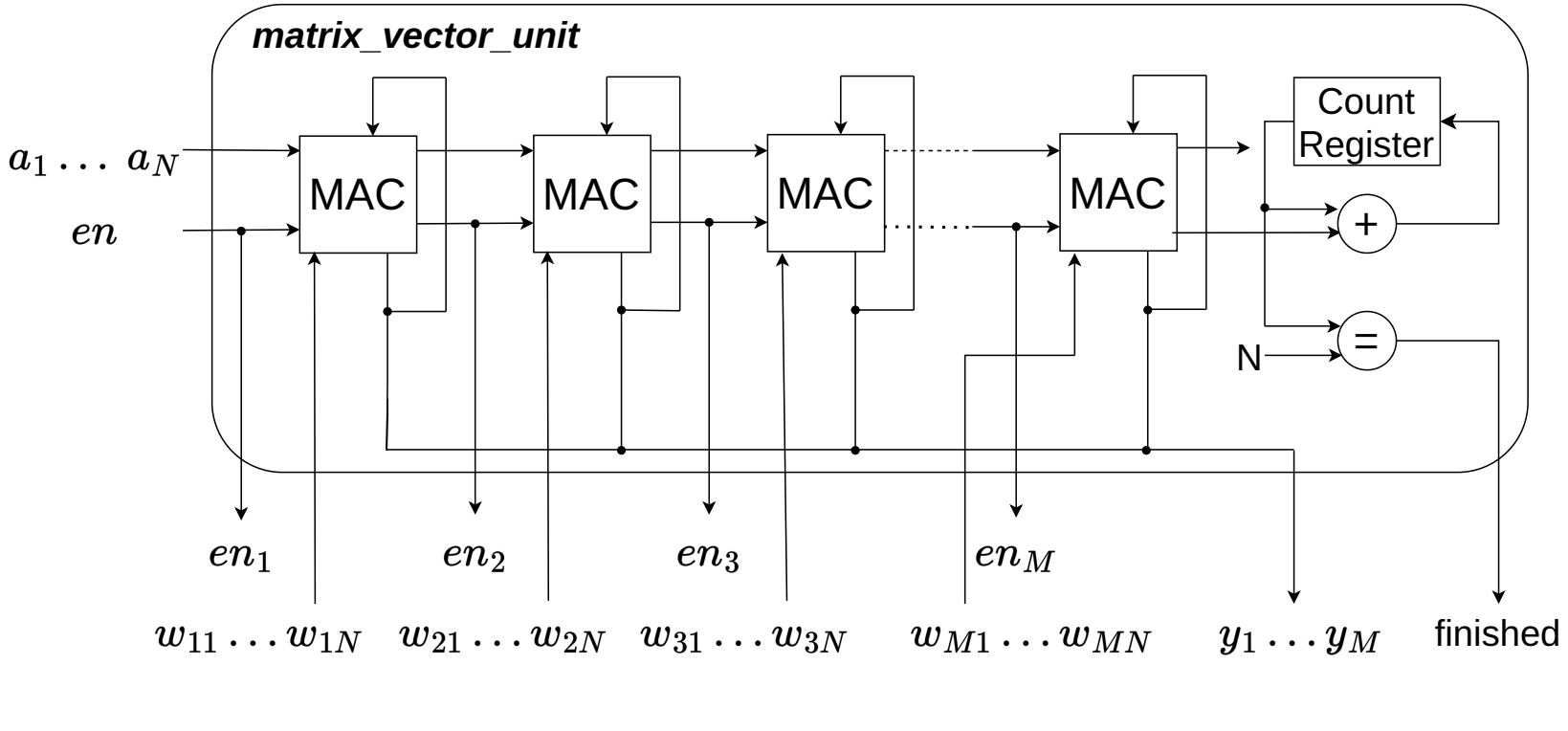


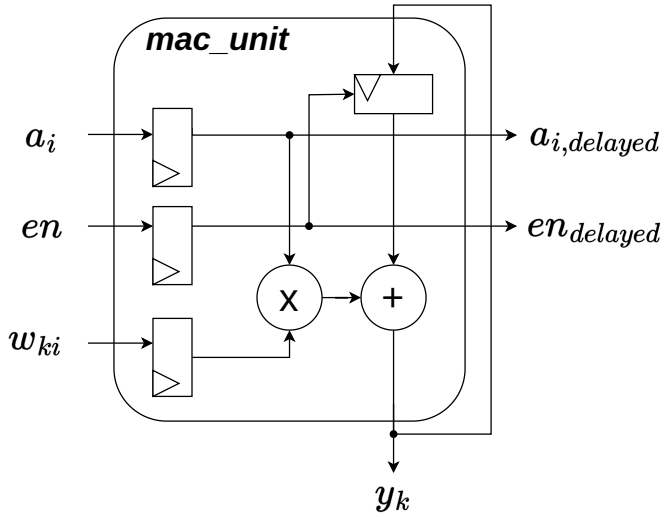
Implementation

Dense-Layers

Matrix-Vector-Unit

Systolic Array for Matrix-Vector-Multiplications



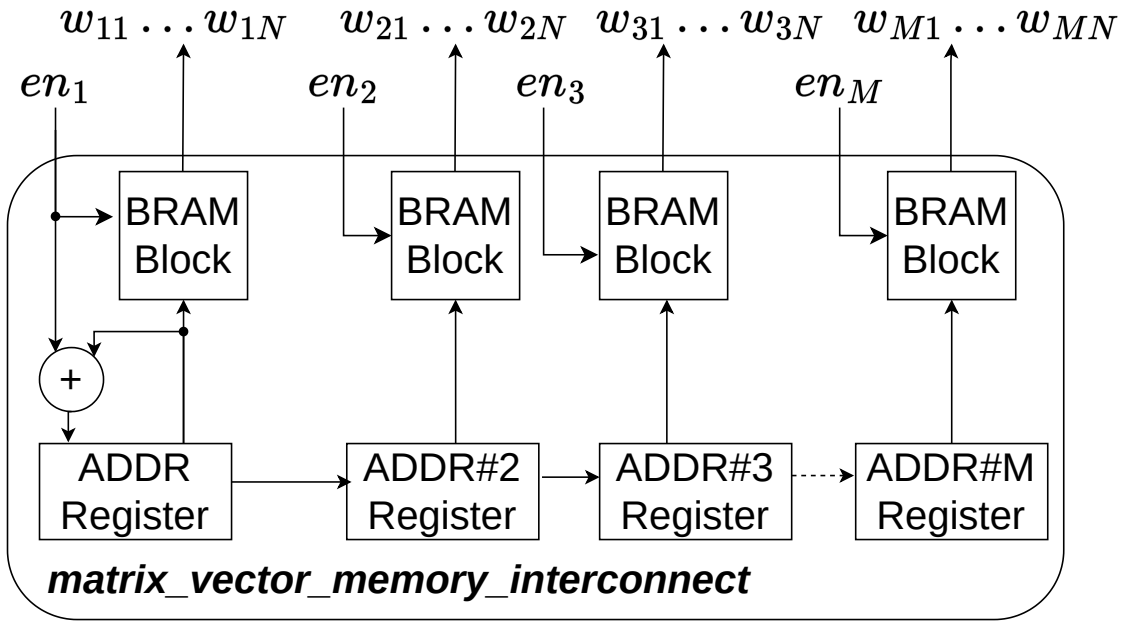


- **Purpose:** Multiplication of weight matrix with activation vector
- Activations and respective enables streamed through MAC-Units

$\Rightarrow$ Unit finished if all *N* activations passed entire pipeline

Matrix-Vector-Unit

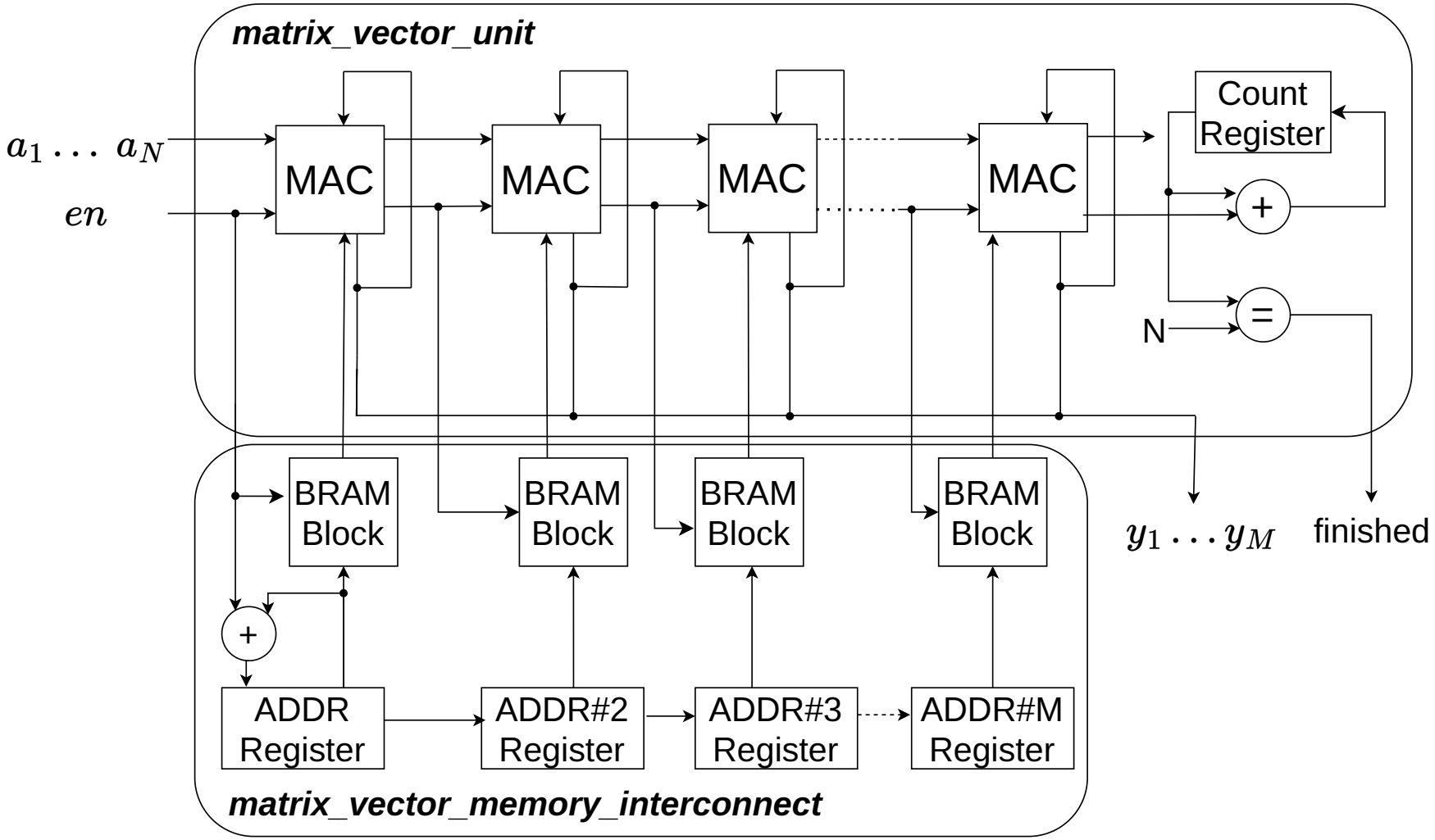
Memory Interconnect



- **Purpose:** Every other weight row delayed by one cycle
- All weight rows stored inside individual BRAM tiles
- Enable signal as read enable for BRAM
- Addressing realized via shift register

Matrix-Vector-Unit

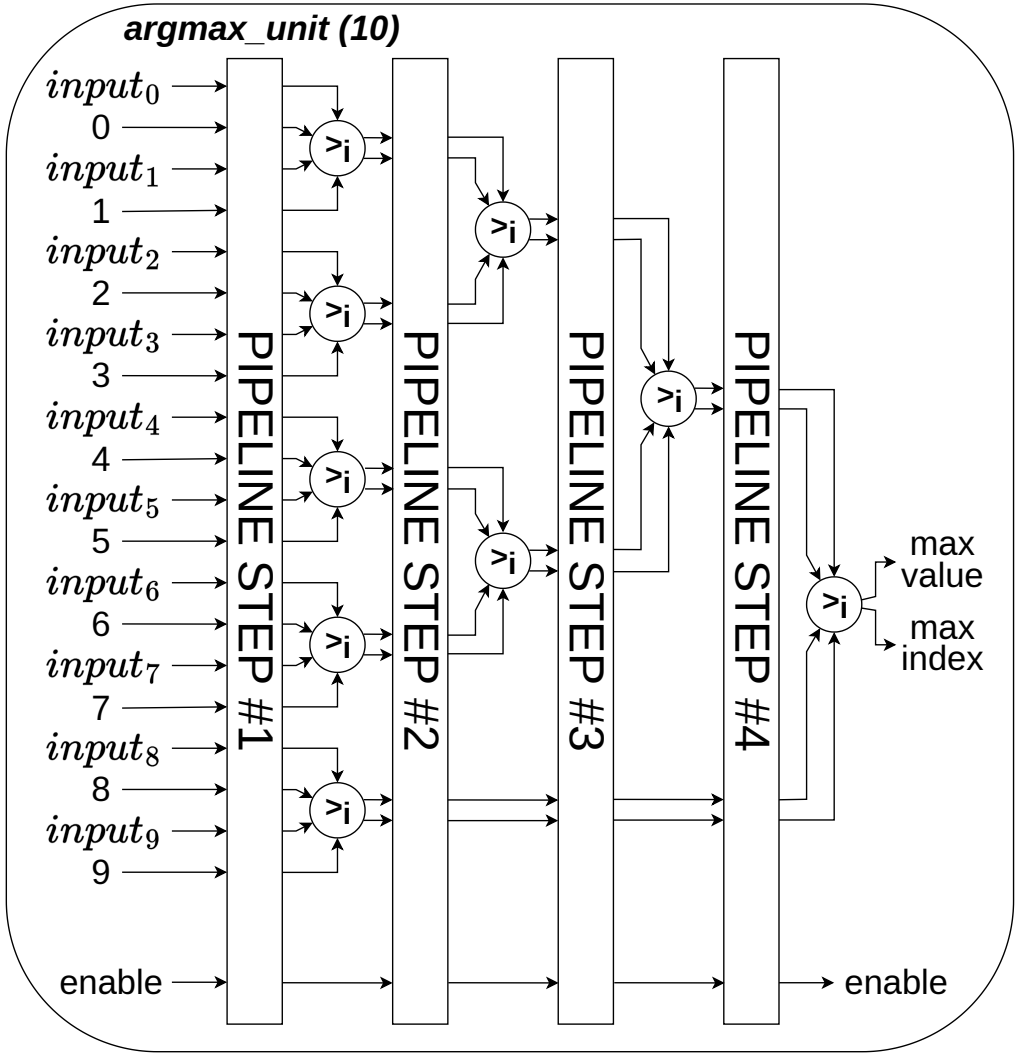
Systolic Array with Memory Interconnect



Argmax Unit

Overview and Design

- **Purpose**: "Probabilities" for each digits → digit label
- **Operation**: Find index of maximum value
- **Implementation**:
  - Generic and pipelined
  - Pipeline each *value* with corresponding *index*
  - $(>_i)$: Output max *value* and corresponding *index*





Implementation

Integrating the Accelerator Core

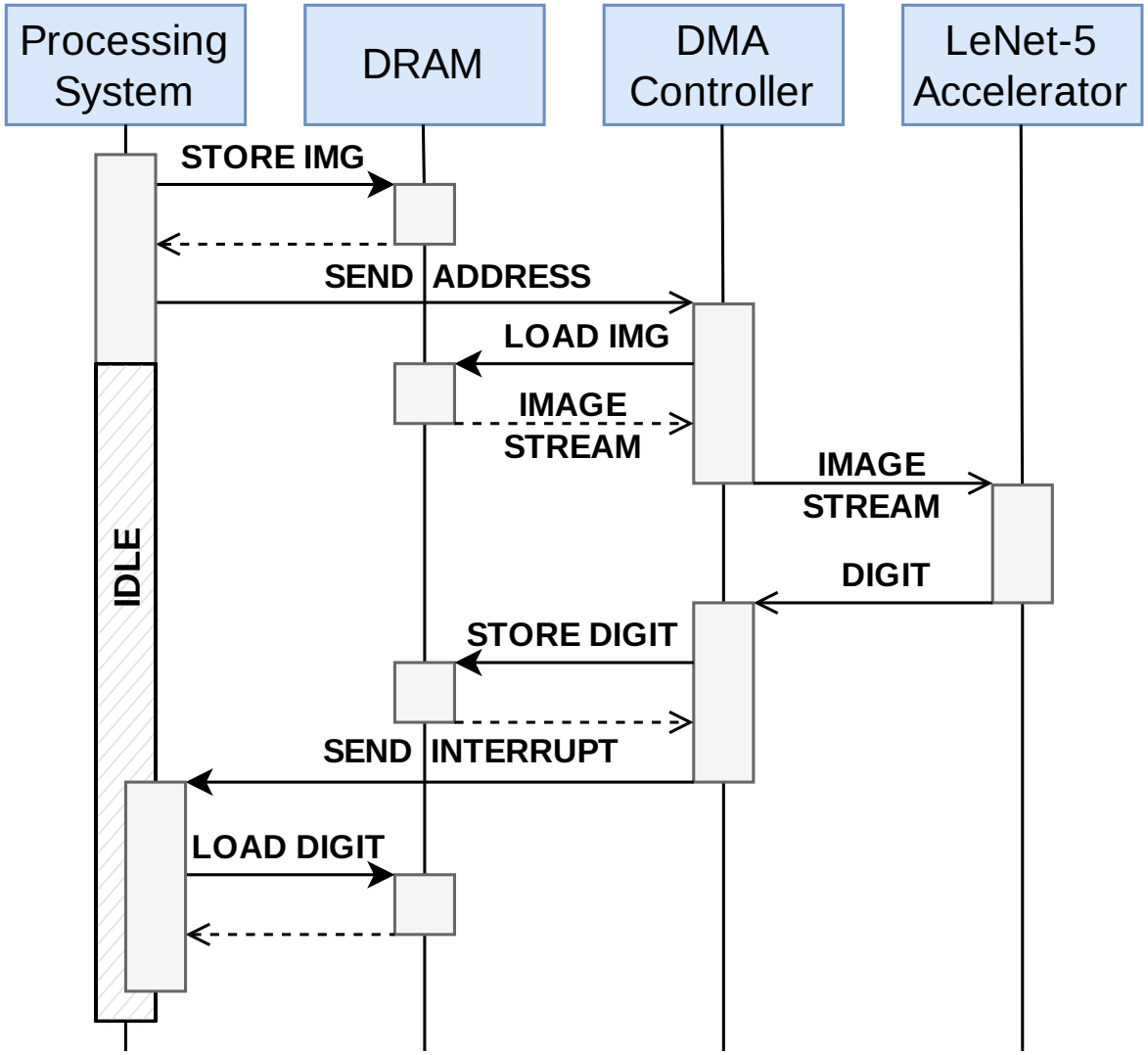
Accelerator Integration

Design Considerations

We target **high efficiency** and **high scalability**:

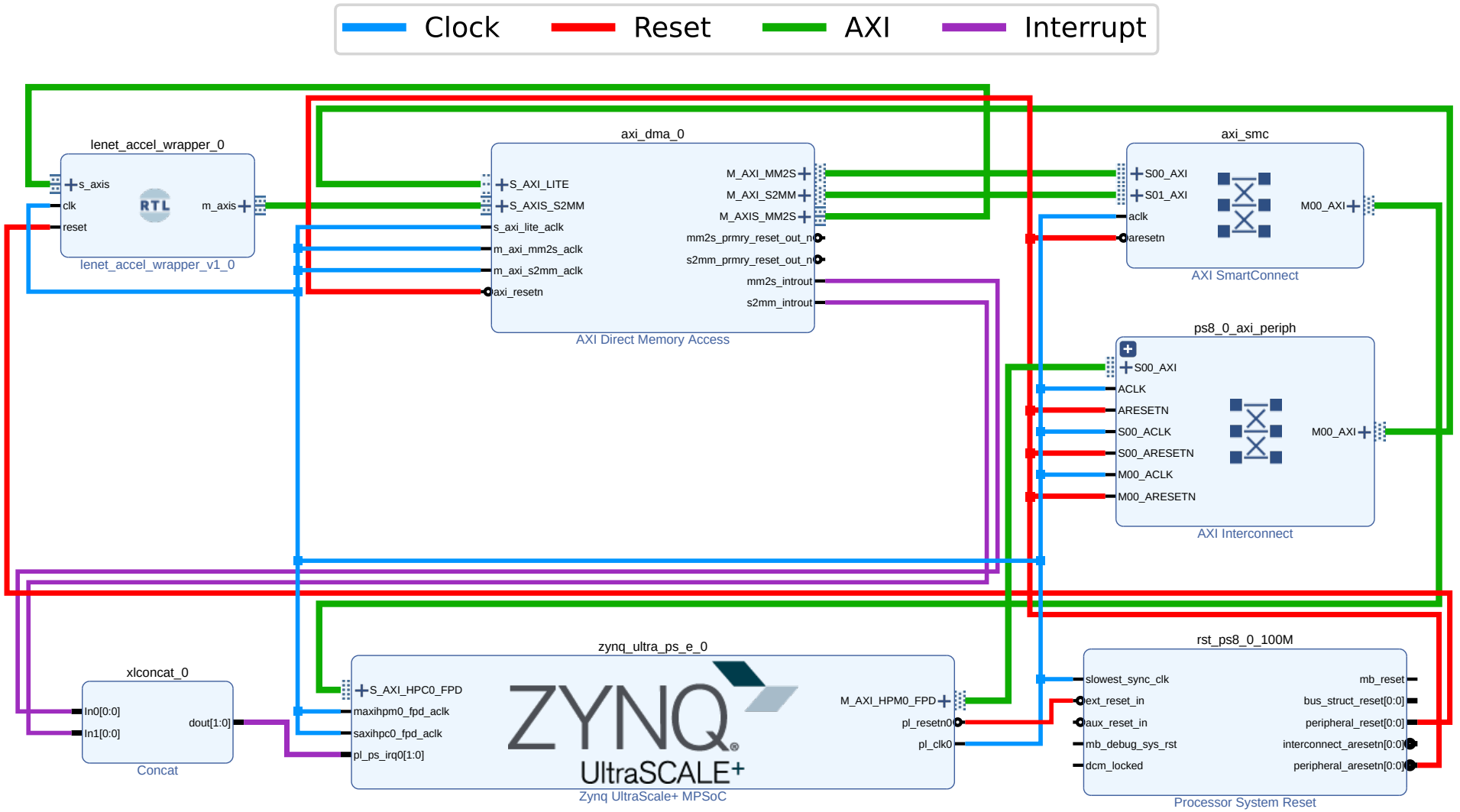
- ~~Programmed I/O~~ vs **DMA**
- ~~Poll-based~~ vs **Interrupt-based**

⇒ **DMA-Controller** already implemented as an **IP-Core** ☺



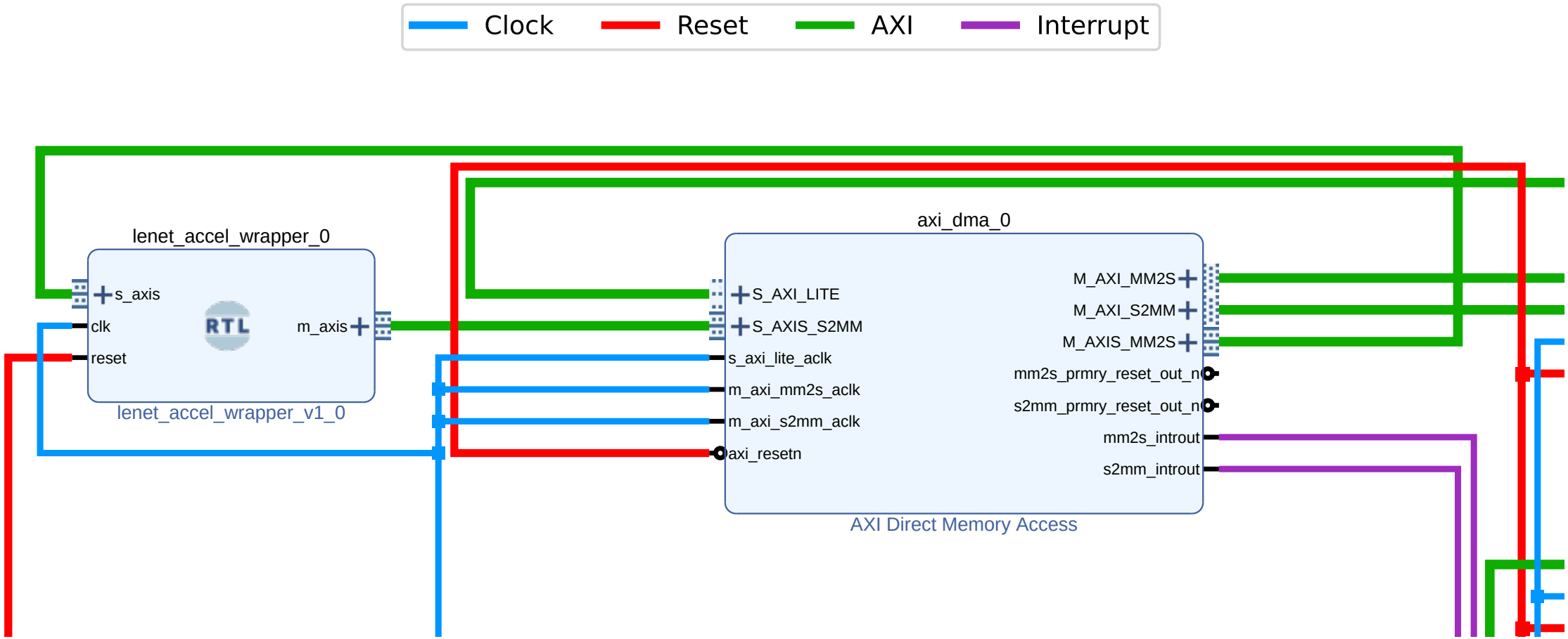
Accelerator Integration

Block Design Overview



Accelerator Integration

Block Design Overview



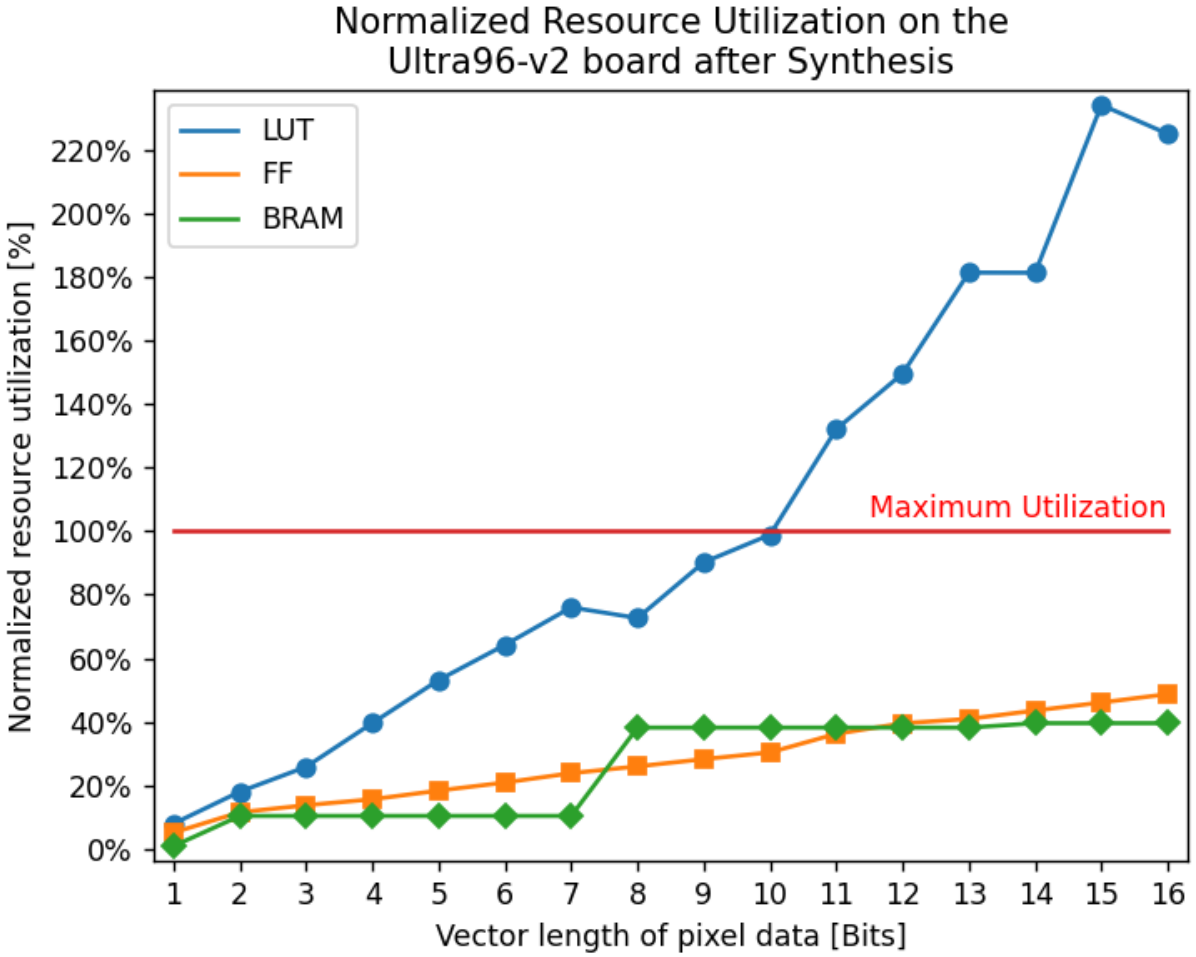
1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

Resource Utilization

Consumption and Scalability of Architecture (w/o DSPs)

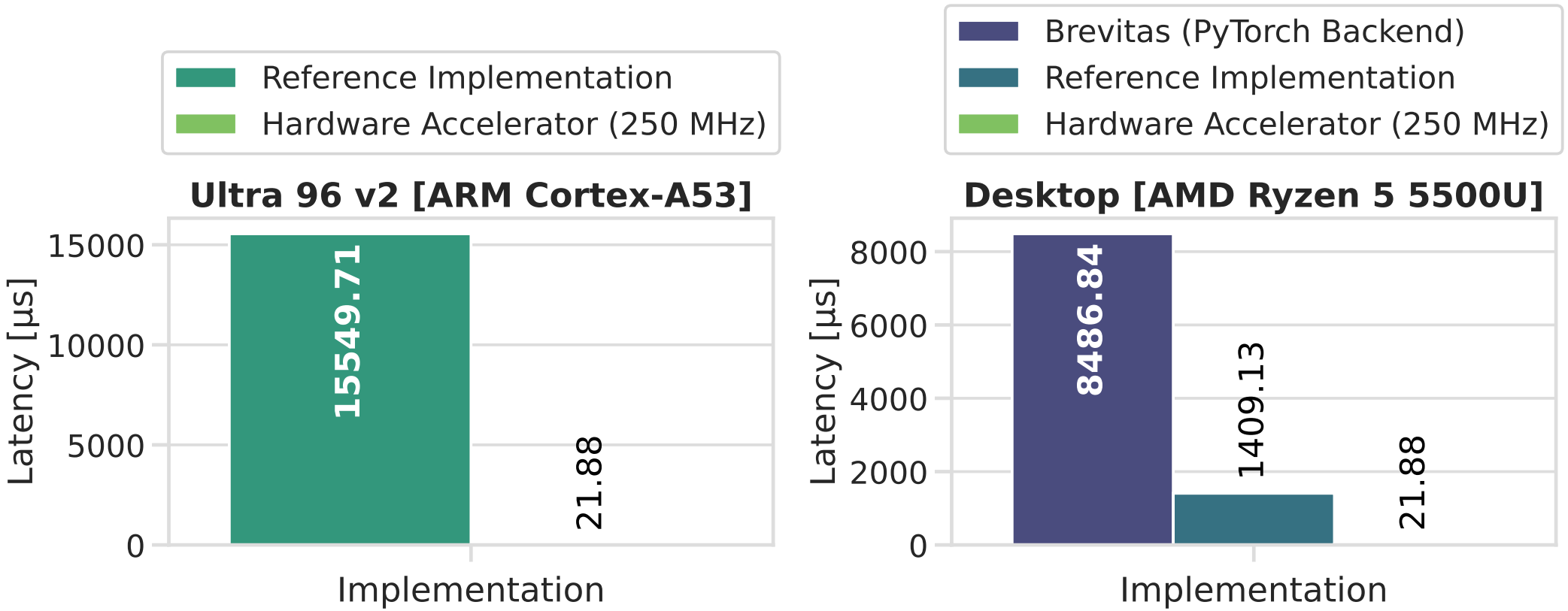
Vivado Synthesis output (*vector_length* = 8):

Site Type	Used	Available	Util%
CLB LUTs	51274	70560	72.67
LUT as Logic	49761	70560	70.52
LUT as Memory	1513	28800	5.25
CLB Registers	36683	141120	25.99
Register as Flip Flop	36683	141120	25.99
Register as Latch	0	141120	0.00
Block Ram Tile	82.5	216	38.19
DSPs	0	360	0.00
Carry8	4721	8820	53.53
F7 Muxes	421	35280	1.19
F8 Muxes	168	17640	0.95
F9 Muxes	0	8820	0.00



Performance Benchmarking

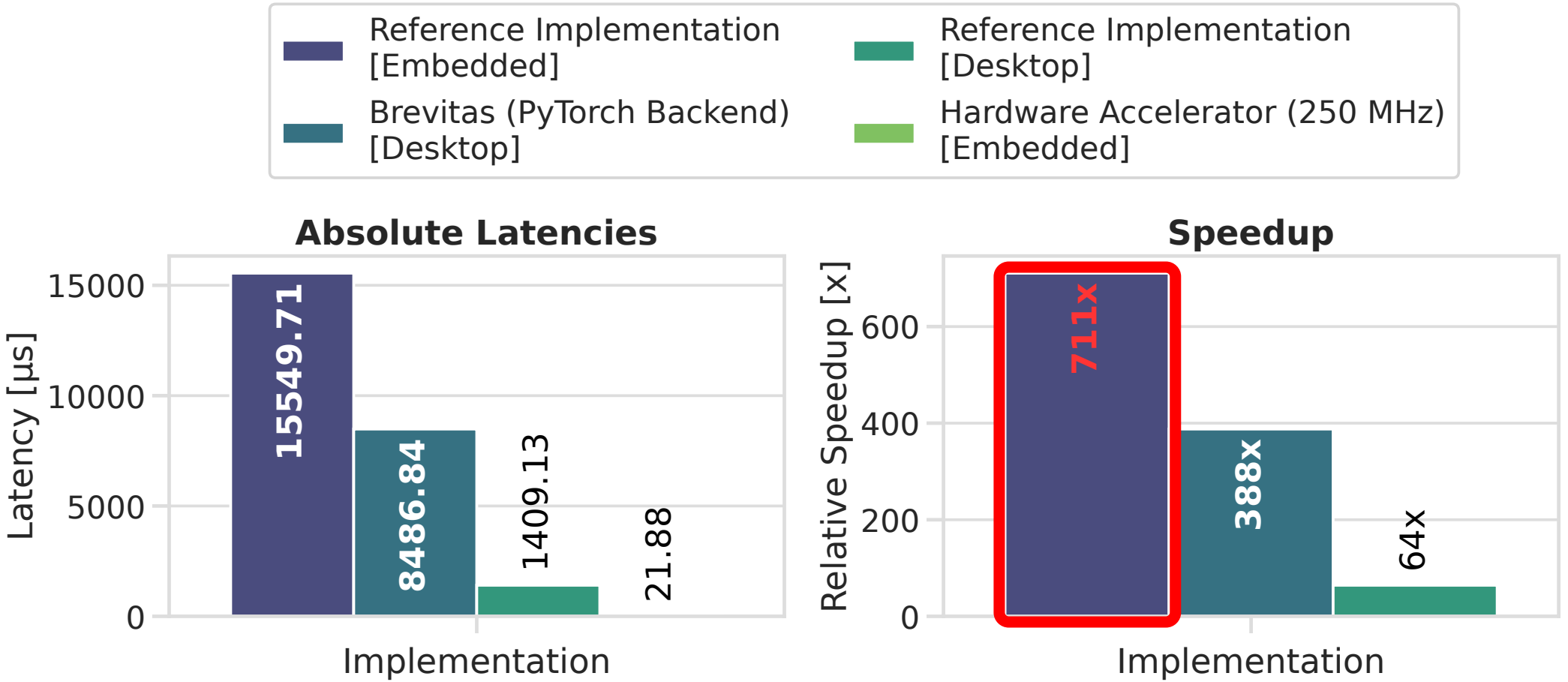
Comparison between Implementations/Systems



(*Restriction*: Batching size is 1)

Performance Benchmarking

Comaprasion between Implementations/Systems

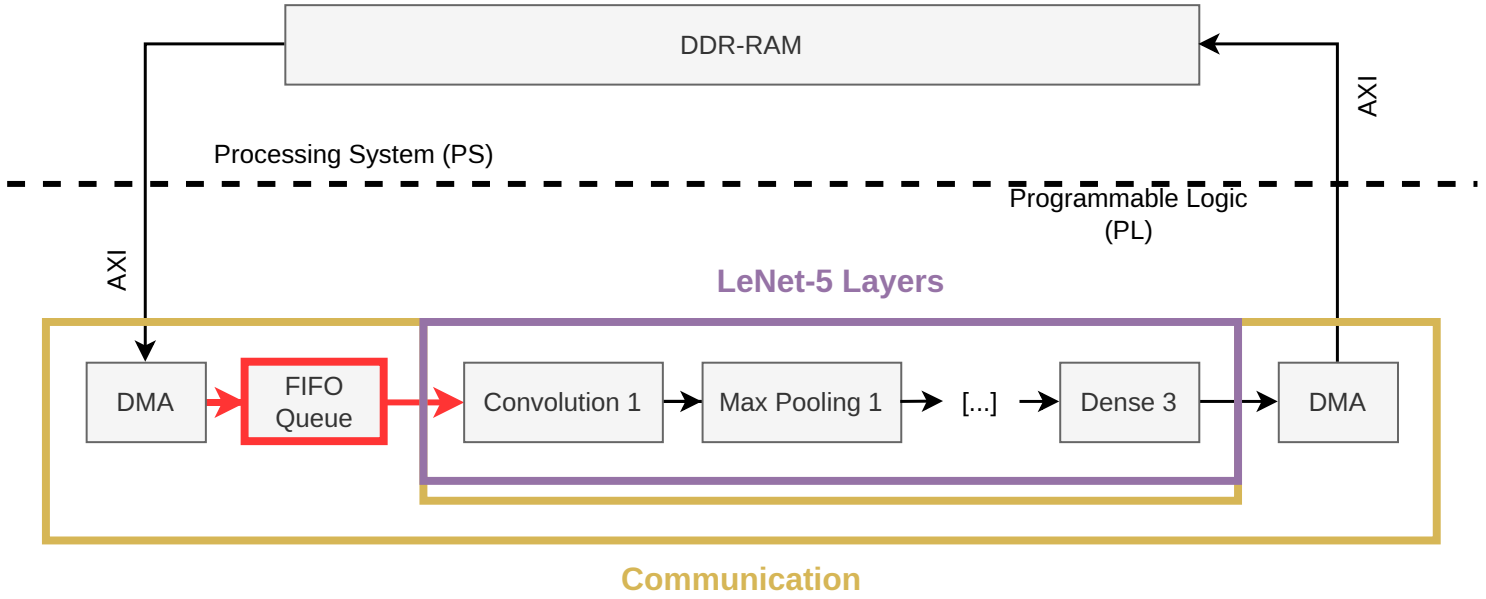


(*Restriction*: Batching size is 1)

Additional Improvements

How could we improve throughput/latency even more?

1. **Queueing**: Waste less time for next image to be transferred



2. **Pipelining**: Fully pipeline the accelerator's stages

3. **Optimize critical paths**: Increase operating frequency

1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

Conclusion

Project Summary

- **Initial Goal:** Hardware accelerated LeNet-5 architecture (high efficient image classification) ✓
- **Implementation:** Partially pipelined and fully generic (easily adaptable)
- **Integration:** DMA-based data transfer with interrupt-based control
- **Performance:** Latency of **21.88µs** (speedup of **711x** compared to reference implementation)!

⇒ **Success!** ☺

Thank you for your attention!

Any questions?

1. Introduction
2. Prerequisites
3. Architecture Overview
4. Implementation
5. Benchmarking
6. Conclusion
7. References

References

[ARM18] L. Aranda, P. Reviriego, and J. A. Maestro. “Protecting Image Processing Pipelines against Configuration Memory Errors in SRAM-Based FPGAs”. In: *Electronics* 7 (Nov. 2018), p. 322.

[Lec+98] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86 (Dec. 1998), pp. 2278–2324. DOI: 10.1109/5.726791.

[SWS19] R. Shi, J. S. Wong, and H. K.-H. So. “High-Throughput Line Buffer Microarchitecture for Arbitrary Sized Streaming Image Processing”. In: *Journal of Imaging* 5.3 (2019). DOI: 10.3390/jimaging5030034.

[Vo+22] Q. H. Vo, N. Linh Le, F. Asim, L.-W. Kim, and C. S. Hong. “A Deep Learning Accelerator Based on a Streaming Architecture for Binary Neural Networks”. In: *IEEE Access* 10 (2022), pp. 21141–21159.